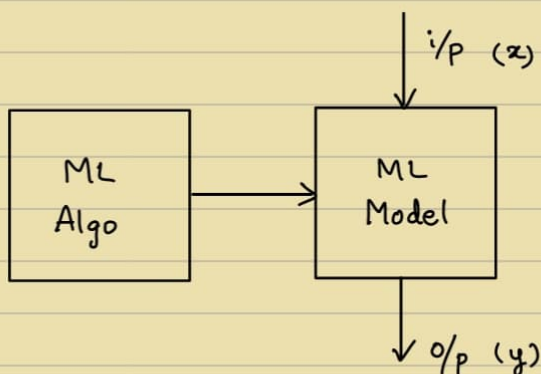


Traditional Programming } $\rightarrow$ Output = ?
Given input & Program



Ex. Sorting, Factorial

Easy for a human to understand and do

- E.g. NLP $\rightarrow$ Human cannot understand & do in the brain
i.e. Voice $\rightarrow$ Wave transform $\rightarrow$ Numbers (data : x_i)

Face Recognition $\rightarrow$ 100s of attributes

i.e. Face $\rightarrow$ Pixels $\rightarrow$ Numbers (data : x_i)

Data : $\{(x_i, y_i)\}$

Q) What makes a chair a chair ?

(i) 4 legs ? But cow has

(ii) Flat surface ? But Desk has

$\vdots$

Exception everywhere , Only brain can know

$\therefore$ ML Model

AI : Blackbox



∴ ML Solution :

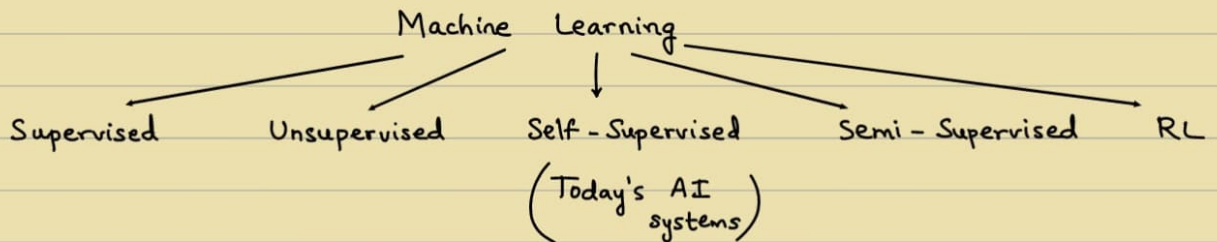
$$y = f(W, x)$$

↳ Learnable parameters

Training : Requires lot of data , GPU & Time

Inference : Testing

→



→ Represent data using Vector

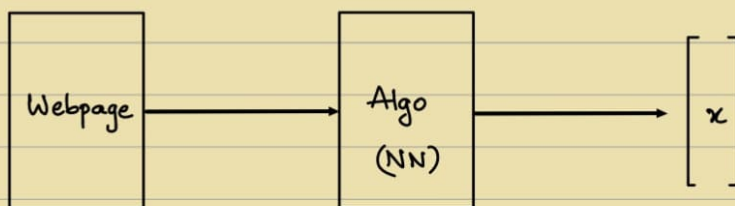
↳ Embedding

$$\begin{bmatrix} x \end{bmatrix}$$

e.g. Webpage → Embed into a fixed dimensional vector which has the essence of the webpage

e.g. Sentiment Analysis , Speech recognition

↳ Represent tweet as a 'd' dimensional vector.
 $x \in \mathbb{R}^d$

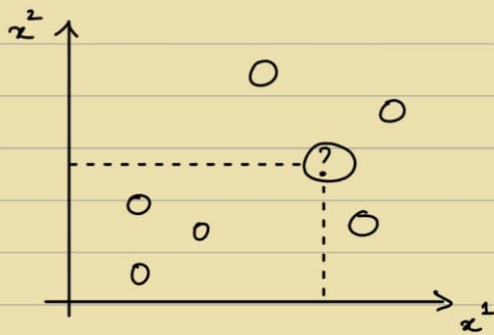


'y' could be scalar / string / Tree / Graph

x is generally vectors / Sequence of vectors

A4 Sheet → Define problem formally

→ Nearest Neighbour Classification :



∴ Lazy algorithm

$$x = \begin{bmatrix} x^1 \\ x^2 \end{bmatrix}$$

Compute distance from all samples
& then sort
& find nearest

Disadvantage : Memory usage ↑
More computation

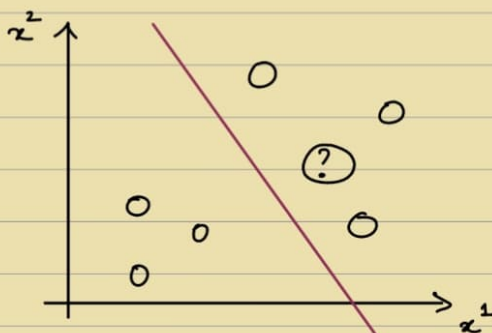
∴ Not efficient

∴ No training

Extension : 3-NN, 5-NN

∴ k-Nearest neighbours - Use Euclidean distance

→ sign($w^T x$) :



$w^T x = 0$ → Hyperplane

$$\begin{bmatrix} w_1 \\ w_2 \\ w_3 \end{bmatrix}^T \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = 0$$

'd' multiplications

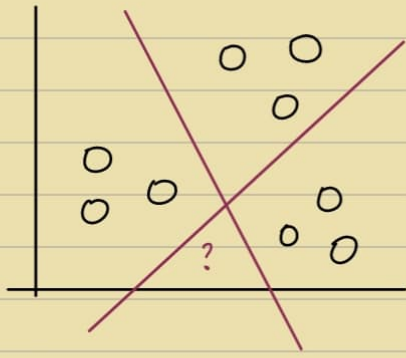
'd-1' additions

$$\Rightarrow w_1 x_1 + w_2 x_2 + w_3 x_3 = 0$$

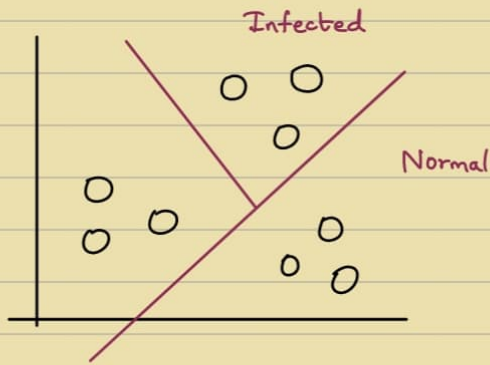
For Multi-class classification, say 3

KNN works.

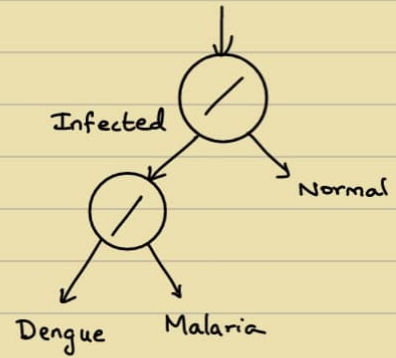
sign($w^T x$) can be extended



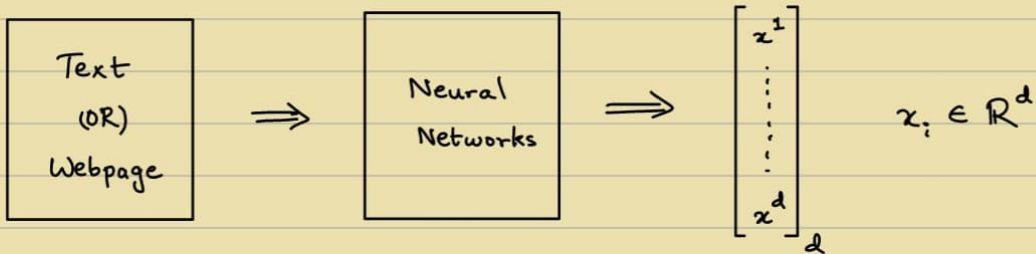
One solution :



1 solution :
Tree-like



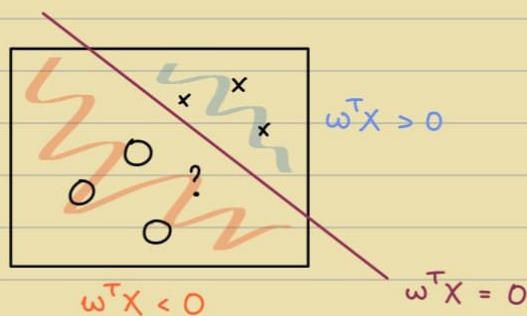
8/1



Similarity

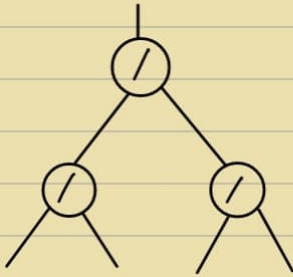
Distance (Triangular Inequality)
[Euclidean - Geometric Intuition]

- Cosine similarity is also another method ($x^T y$)



→ Multi-class classifiers :

(1) Hierarchical



e.g. k classes , $\log(k)$ classifiers

Cons : Many calculations , Not efficient

Pros : Lesser Inference time

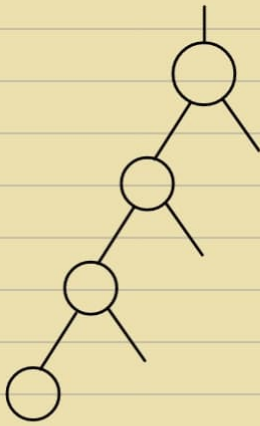
(2) Majority Voting :

$k C_2$ classifiers , k : no. of classes

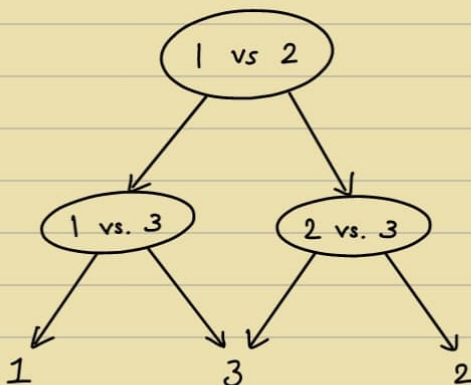
Computational complexity : k^2

Accuracy - wise

(3) 1 vs. Rest

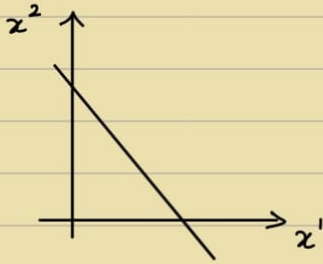


• Optimization : DAG (Directed Acyclic Graph)



- $w^T X = 0$

$$w_1 x^1 + w_2 x^2 + w_3 = 0$$



$$\begin{bmatrix} w_1 \\ w_2 \\ w_3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ 1 \end{bmatrix}$$

- x vector is being improved as it passes through each layer

$$\begin{bmatrix} x \end{bmatrix} \xRightarrow{\phi} \begin{bmatrix} x' \end{bmatrix}$$

$$X' = WX$$

↪ Linear Transformation

$$X_d \rightarrow W_{m \times d}$$

$m \ll d$: Dimensional Reduction
(e.g. PCA)

$m > d$: New enriched features
(Weighted combination)
e.g. FC, MLP

Numerically useful :

$$\mu = 0 \text{ \& \; } \sigma^2 = 1 \quad [\text{Standard Scalar}]$$

[OR]

Scale to range $[0, 1]$

$$\|L_2\| = 1$$

Normalization

$$\begin{bmatrix} 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

W

e.g. Reducing :

$$\{10, 25, -5, 7.5, 10\}$$

$x_1 \quad x_2 \quad x_3 \quad x_4 \quad x_5$

$$x' = \frac{x - \min}{\max - \min} \implies [0, 1]$$

$$x' = \frac{x - \mu}{\sigma} \implies \mu = 0 \text{ \& \; } \sigma^2 = 1$$

Stable Algorithms $\because$ Numbers are well-behaved
& $\therefore$ Better result

- Normalizing vectors :

$$x = \begin{bmatrix} 2 \\ 1 \\ 1.5 \end{bmatrix}$$

One method $\rightarrow \frac{x}{\|x\|}$

$$X_1, X_2, \dots, X_5$$

$\downarrow$

$$\bar{X}_1, \bar{X}_2, \dots, \bar{X}_5$$

$\downarrow$

$$\bar{X}_1 - \mu, \bar{X}_2 - \mu, \dots, \bar{X}_5 - \mu$$

$$\mu = \frac{X_1 + X_2 + \dots + X_5}{5}$$

For variance $\Rightarrow$ Divide by σ^2

But covariance can exist.

$$\Sigma^{-1} [\bar{X}_1 - \mu]$$

$\swarrow$
Covariance Matrix

$$\mu = \frac{1}{N} \sum_{i=1}^N X_i$$

$$\Sigma = \frac{1}{N} \sum_{i=1}^N [X_i - \mu][X_i - \mu]^T$$

$$\Sigma_{d \times d}^{-1}, X_{i \, d \times 1}, \mu_{d \times 1}$$

$\rightarrow$ Evaluation Metrics :

Accuracy
Confusion Matrix
Precision
Recall
AP
F-score

$\left. \begin{array}{l} \text{Confusion Matrix} \\ \text{Precision} \\ \text{Recall} \end{array} \right\} \text{Classification}$

$\left. \begin{array}{l} \text{AP} \\ \text{F-score} \end{array} \right\} \text{Ranking / Retrieval}$

$$\text{Accuracy} : \frac{\text{Total correct}}{\text{Total}}$$

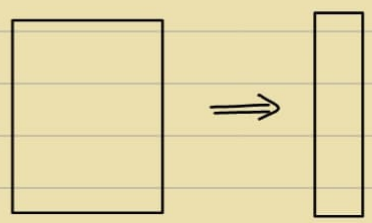
$\rightarrow$ Reduction in KNN :

$$\text{Euclidian Distance b/w } X \text{ and } Y = [X_1 - X_2]^T [X_1 - X_2]$$

$$\begin{matrix} [X_1 - X_2]^T & D & [X_1 - X_2] \\ 1 \times d & d \times d & d \times 1 \end{matrix}$$

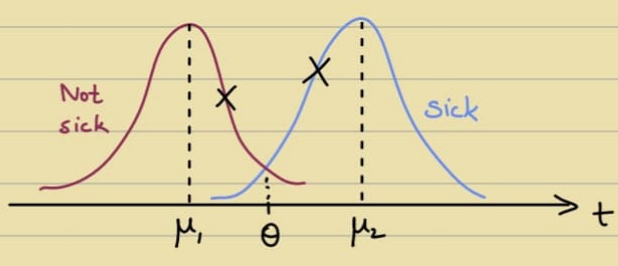
$\rightarrow$ Diagonal Matrix, $D = \Sigma^{-1}$

{ gyan.iit.ac.in }

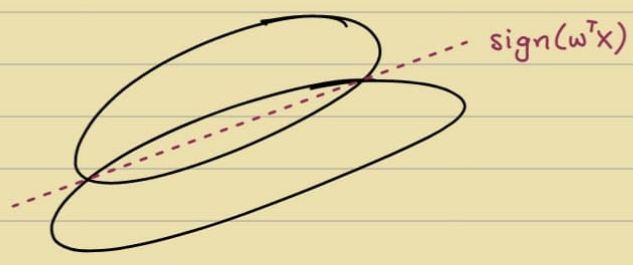


$$D = \{x_i, y_i\}$$
$$i = 1, 2, \dots, N$$
$$x \in \mathbb{R}^d$$
$$y \in \mathbb{R}$$

$$\text{sign}(w^T x) \leftarrow f(w, x)$$

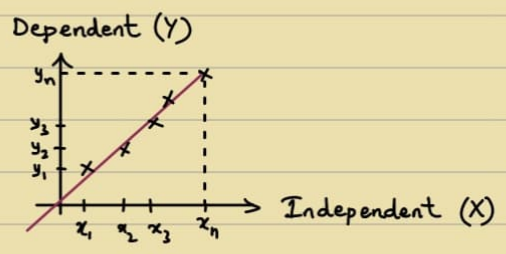


$$P(\text{sick} | x) \geq P(\text{Not sick} | x)$$
$$\therefore \text{Sick, else not sick}$$



- Spec. of an ML Software: Accuracy & Precision
[98% accurate 95% of the time]
PAC: Probablittically Approximate.

→ Regression:
 $y \in \mathbb{R}$



Do well on } $\Rightarrow$ Generalisation
Unseen data

$$\text{MSE Loss} : (y_1 - \hat{y}_1)^2 + (y_2 - \hat{y}_2)^2 + \dots + (y_{n-1} - \hat{y}_{n-1})^2$$

$\hat{y}$: Predicted

$$\text{minimise MSE loss by varying } w \Rightarrow \min_w L(y, \hat{y})$$
$$w^* = \arg \min_w L(y, \hat{y})$$

$\nwarrow w^T x$
 $f(w, x)$

$$\Rightarrow w^* = \arg \min_w \frac{1}{N} \cdot \sum_{i=1}^N (y_i - w^T x_i)^2$$

$$y = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix} \begin{bmatrix} x_1 \\ 1 \end{bmatrix}$$

[Discrete optimization]

Loss function should be differentiable.

$$y = w_1 x_1 + w_2$$

$$\therefore w^* = \frac{1}{N} \cdot \arg \min_w \left(\sum_{i=1}^N (y_i - w^T x_i)^2 \right)$$

Accuracy cannot be taken as loss function $\therefore$ Non-differentiable

→ Closed Form Solution:

$$\hat{y}_1 = w^1 x_1^1 + w^2 x_1^2 + w^3 x_1^3 + w^4 1$$

$$\hat{y}_2 = w^1 x_2^1 + w^2 x_2^2 + w^3 x_2^3 + w^4 1$$

$$\hat{y}_3 = w^1 x_3^1 + w^2 x_3^2 + w^3 x_3^3 + w^4 1$$

$$\begin{bmatrix} y \\ \vdots \\ y \end{bmatrix}_{n \times 1} = \begin{bmatrix} X \\ \vdots \\ X \end{bmatrix}_{n \times d} \begin{bmatrix} w \\ \vdots \\ w \end{bmatrix}_{d \times 1}$$

$$\frac{1}{N} [Y - \hat{Y}]^T [Y - \hat{Y}]$$

$$\Rightarrow [Y - Xw]^T [Y - Xw]$$

$$\Rightarrow Y^T Y + (Xw)^T (Xw) - 2(Xw)^T Y$$

$$\Rightarrow Y^T Y + w^T X^T X w - 2w^T X^T Y$$

$$\frac{\partial}{\partial w} (Y^T Y + w^T X^T X w - 2w^T X^T Y)$$

$$= \frac{\partial}{\partial w} (w^T X^T X w) - 2 X^T Y$$

$$= 2 X^T X w - 2 X^T Y \quad \underline{\underline{\text{set } 0}}$$

$$X^T X w = X^T Y$$

$$\Rightarrow w = \underbrace{(X^T X)^{-1}}_{d \times d} \underbrace{X^T}_{d \times n} \underbrace{Y}_{n \times 1}$$

$w^T A w \rightarrow$ Quadratic form in matrices

$$\frac{\partial}{\partial w} (w^T A w) = 2 A w$$

$$\begin{bmatrix} w : d \times 1 \\ A : d \times d \end{bmatrix}$$

$$\{A^T B = B^T A\}$$

→ Gradient descent:

Start with w^0

Iterate (To minimize J)

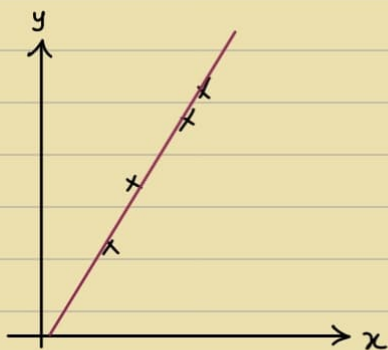
$$w^{k+1} \leftarrow w^k - (\nabla J)(lr)$$

(lr : Learning Rate)

$$\min_w J = \min_w \frac{1}{N} \sum_{i=1}^N (y_i - w^T x_i)^2$$

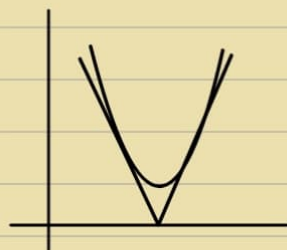
$$\nabla J = \frac{2}{N} \sum_{i=1}^N (y_i - w^T x_i) (-x_i)$$

19/1



$$y = w^T x + w^0$$

$$\frac{1}{N} \cdot \sum_{i=1}^N (\hat{y}_i - y_i)^2$$



$$y = w^T X = f(w, X)$$

$$= \begin{bmatrix} w_1 \\ w_2 \end{bmatrix} \begin{bmatrix} x \\ 1 \end{bmatrix}$$

$$\left. \begin{array}{l} y_1 = w^T x_1 + w^0 \\ y_2 = w^T x_2 + w^0 \\ \vdots \\ y_N = w^T x_N + w^0 \end{array} \right\} \Rightarrow \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix} = \begin{bmatrix} x_1 & 1 \\ x_2 & 1 \\ \vdots & \vdots \\ x_N & 1 \end{bmatrix} \begin{bmatrix} w^1 \\ w^0 \end{bmatrix}$$

$$\Rightarrow Y = Xw$$

$$\begin{array}{l} [Y - \hat{Y}]^T [Y - \hat{Y}] \\ [Y - Xw]^T [Y - Xw] \end{array}$$

- Closed Form Sol. :

$$w^* = (X^T X)^{-1} X^T Y$$

- Gradient Descent Sol. :

(i) Initialize w^0

$$(ii) w^{k+1} \leftarrow w^k - \eta \nabla J$$

$$\min_w J(w) = \min_w \frac{1}{N} \sum_{i=1}^N (y_i - w^T x_i)^2$$

$$\nabla J = \frac{2}{N} \sum_{i=1}^N (y_i - w^T x_i) (-x_i)$$

→ Regularisation :

Some features are not as useful as the others.

$$J = f(w) + \lambda g(x)$$

$$J = f(w) + \lambda \|w\|$$

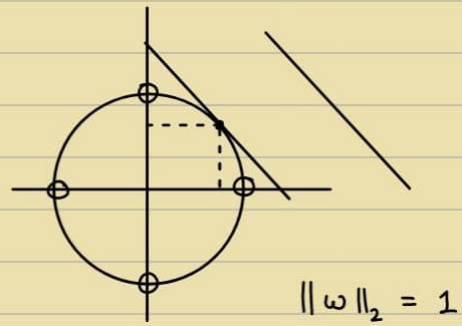
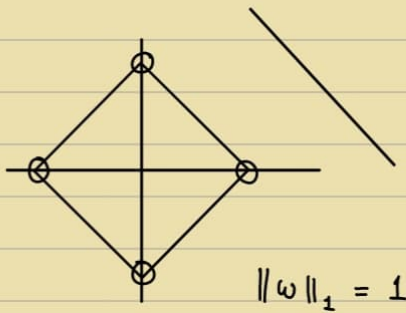
Regularize with L_2, L_1 (ideally L_0) norms

$$\|x\| = \left(\sum_{i=1}^d |x_i|^p \right)^{1/p}$$

$$L_1 \ (d=2) : \|x\|_1 = |x_1| + |x_2|$$

$$L_2 \ (d=2) : \|x\|_2 = \sqrt{x_1^2 + x_2^2}$$

$\left\{ \begin{array}{l} L_0 \text{ norm : Counts the non-zero elements in the vector} \\ L_\infty \text{ norm : Max. Abs. value of the vector} \leftarrow \text{max. norm} \end{array} \right.$
 Pseudo-norms




 Better in eliminating Sparsity

$$\min J(w) \quad \text{s.t.} \quad \|w\| = 1$$

- Ridge Regression :

Linear Regression normalized with L_2 Norm

$$J = \sum_{i=1}^N (y_i - w^T x_i)^2 + \lambda \cdot \sum_{j=1}^d w_j^2$$

$$= \sum_{i=1}^N (y_i - w^T x_i)^2 + \lambda \cdot \|w\|_2^2$$

$$J(w) = \frac{1}{N} (Y^T Y + w^T X^T X w - 2 w^T (X^T Y) + \lambda w^T w)$$

$$\text{Optimizing} \Rightarrow w = (X^T X + \lambda I)^{-1} X^T Y$$

- Lasso Regression :

Linear Regression normalized with l_1 Norm

$$J = \sum_{i=1}^N (y_i - w^T x_i)^2 + \lambda \cdot \sum_{j=1}^d |w_j|$$

$$= \sum_{i=1}^N (y_i - w^T x_i)^2 + \lambda \cdot \|w\|_1^2$$

Solution to Lasso - Outside the scope of course
Optimization leads to zero of irrelevant features.

→ Eigenvalues & Eigenvectors :

$$Av = \lambda v$$

$\swarrow$ Eigenvector (v_i) ← Normalized to unit norm
 $\searrow$ Eigenvalues (λ_i) ← Sorted in decreasing order

Max. $w^T A w$ s.t. $w^T w = 1$ ($\|w\|_2 = 1$)

Using Lagrangian (λ) :

$$J(w, \lambda) = w^T A w - \lambda (w^T w - 1)$$

Differentiate w.r.t w :

$$Aw = \lambda w$$

$w \rightarrow$ eigenvalue corresponding to the largest eigenvalue.

$$\therefore \max. w^T A w \text{ s.t. } w^T w = 1$$

$$\Rightarrow \max. w^T \lambda w$$

$$\Rightarrow \max. w^T w \cdot \lambda$$

$$\Rightarrow \max. \lambda \leftarrow \therefore \text{Pick largest } \lambda$$

$$Ax = 0 : \text{Homogenous}$$

$$\hookrightarrow m \times n$$

$$\text{Min. } \|Ax\| \text{ s.t. } \|x\| = 1$$

$$\Rightarrow \min_x x^T A^T A x - \lambda (x^T x - 1)$$

$$\Rightarrow 2AA^T x = 2\lambda x$$

$$\Rightarrow AA^T x = \lambda x$$

Assuming $y_i = mx_i + c$ form, derive update rule for m & c

Iterative : Better

Iterative & Closed Form $\rightarrow$ Same answer

More Memory

Inverse is computationally expensive

But Parallelism

Processing in batches $\rightarrow$ Stochastic Gradient Descent

• e.g. $\alpha_i \in [0, 1]$

0 : non-confident

1 : confident

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N \alpha_i (y_i - w^T x_i)^2$$

- Closed Form

$$\begin{bmatrix} Y \end{bmatrix}_{n \times 1} = \begin{bmatrix} \alpha \end{bmatrix}_{n \times n} \times \begin{bmatrix} w \end{bmatrix}_{d \times 1}$$

$\searrow$ Diagonal Matrix

- Gradient Descent :

$$w^{k+1} \leftarrow w^k - \nabla J$$

$$\min_w J = \min_w \frac{1}{N} \sum_{i=1}^N \alpha_i (y_i - w^T x_i)^2$$

$$\Rightarrow \nabla J = \frac{2}{N} \sum_{i=1}^N \alpha_i (y_i - w^T x_i) (-x_i)$$

22/1

$$\begin{bmatrix} \vdots \\ \vdots \\ \vdots \end{bmatrix} \in \mathbb{R}^d$$

N

$$X = \begin{bmatrix} \vdots & \vdots & \vdots \end{bmatrix}$$

$N \times d$

$$X^T X : d \times d$$

$(N > d)$

$$\Sigma = \frac{1}{N} \sum_{i=1}^N [x_i - \mu][x_i - \mu]^T$$

$$\mu = \frac{1}{N} \sum_{i=1}^N x_i$$

$$(d \times 1) \times (1 \times d) \equiv (d \times d) \leftarrow \text{Rank} = 1$$

→ One-hot representation:

$$\begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix} \quad \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \\ \vdots \\ 0 \end{bmatrix}$$

• Rank of matrix after adding noise → no. of non-zero eigen values.

$$A : \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \xrightarrow{\substack{R_2 : R_2 - R_1 \\ R_3 : R_3 - R_1}} \text{Rank} = 2$$

$$A + \epsilon B, \text{ if } \epsilon = 1 : \text{Rank} = 3$$

$$\text{if } \epsilon = 0.001 : \text{Rank} = 2$$

$$\text{Rank of data matrix} : \min(N, d)$$

- Numerical rank vs. Practical rank

Learning is easier when low rank structure, else classifier not very accurate

$$A_v = \lambda_v$$

↳ $N \times N$ [Positive Semidefinite]
($x^T A x = 0$)

Eigen decomposition : $A = \sum_{i=1}^d \lambda_i v_i v_i^T$

$$A = U \Lambda U^T$$

Singular Value Decomposition (SVD)

$$A = U \Sigma V^T$$

5×3 5×3 3×3 3×3
 3×5 3×3 3×3 3×3

$$A = U D V^T$$

$$\Rightarrow AA^T = (UDV^T)(VD^T U^T) \\ = UD^2 U^T$$

$$A^T A = (VD^T U^T)(UDV^T) \\ = VD^2 V^T$$

$A_{N \times N}$ has closest low rank p , $p < N$, say A'

$$A' = \arg \min_M \|A - M\|$$

rank of M is p .

Solve using SVD :

$$A \rightarrow U D V^T$$

$A' \leftarrow$ 

[Look at the data &
Analyze it first!]

Tiny noise : Rank - 2 → 100
 ↓ ↓
 Practical Numerical

Sorted in descending order
(eigen spectrum)



$$A = X^T X$$

$$\Rightarrow A^T = (X^T X)^T = X^T X^T{}^T = X^T X = A$$

A : Symmetric

$$z^T (X^T X) z = (Xz)^T (Xz)$$

$\therefore X^T X$ is positively semi-definite.

$\therefore A$ is SPD.

$A_{d \times d}$

- Using SVD to find inverse

$$A = U D V^T$$

$$\Rightarrow A^{-1} = (V^T)^{-1} (D^{-1}) (U^{-1})$$

$$= V (D^{-1}) U^T$$

- $\text{Min}_Y \sum_{i=1}^N (x_i - Y)^T (x_i - Y)$

$$\Rightarrow \frac{\partial}{\partial Y} \left(\sum_{i=1}^N x_i^T x_i + Y^T Y - 2 x_i^T Y \right) \stackrel{\text{set}}{=} 0$$

$$\Rightarrow 2Y - 2x_i = 0$$

$$\therefore Y = \frac{1}{N} \sum_{i=1}^N x_i$$

→ PCA:

① $\mu = ?$

② $\bar{x}_i = x_i - \mu$

③ $\Sigma = \frac{1}{N} \sum \bar{x}_i \bar{x}_i^T$

$$|\Sigma - \lambda I| = 0$$

$$v_i = ?$$

④ $z_i = v_i^T \bar{x}_i$

⑤ $\hat{x}_i = \mu + z_i v_i$

$$(X^T X)^{-1} X^T y$$

$$\omega^{k+1} \leftarrow \omega^k - \frac{2}{N} \sum_{i=1}^N (y - \omega^T x_i)(-x_i)$$

$$L_1: |x_1| + |x_2|$$

$$L_2: \sqrt{x_1^2 + x_2^2} \rightarrow \text{Ridge}$$
$$(X^T X + \lambda I)^{-1} X^T y$$

SVD:

$$M = U D V^T$$

2/1

$$x_1, x_2, \dots, x_N ; x_i \in \mathbb{R}^d$$



$$x_i' = W \cdot x_i$$

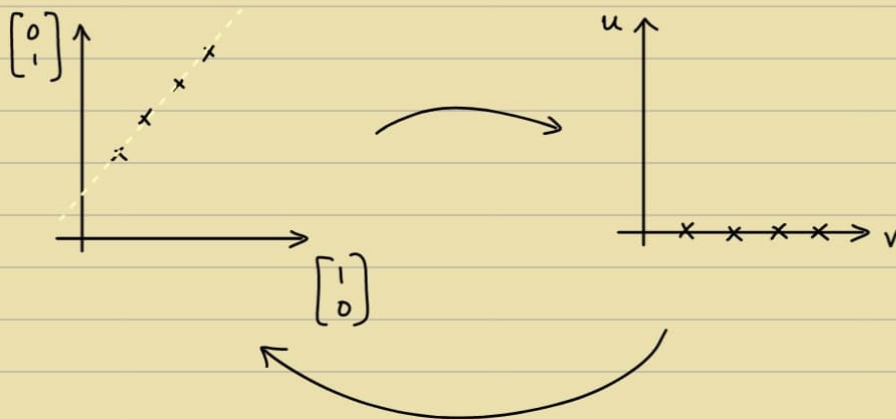
$$\mathbb{R}^{d'}$$



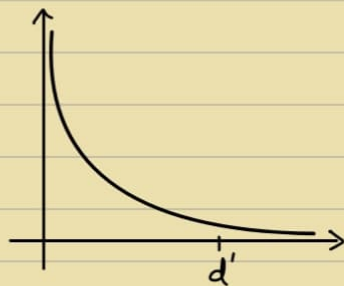
$$d' \times d, \text{ E. Vector of } \Sigma = \frac{1}{N} \sum_{i=1}^N [x_i - \mu][x_i - \mu]^T$$

$$\mu = \frac{1}{N} \cdot \sum_{i=1}^N x_i$$

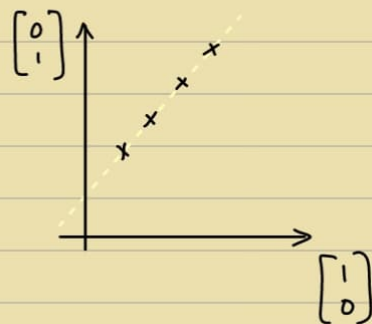
Dimensionality Reduction $\Rightarrow$ Compression



$d' \times d \Rightarrow d' \times d' \therefore$ Possible, but variations around the line are lost (tiny)



Eigenvalues



$$d : u^T x_i$$

$$\text{Minimise } -u^T \Sigma u \quad \text{s.t. } u^T u = 1$$

$$\text{Maximise } u^T \Sigma u \quad \text{s.t. } u^T u = 1$$

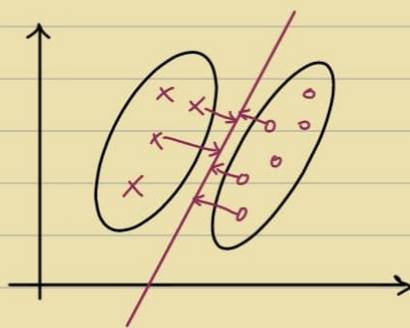
PCA : KL Transform - Basis changes [Unsupervised]

DCT : Basis is Static - LDA [Supervised]

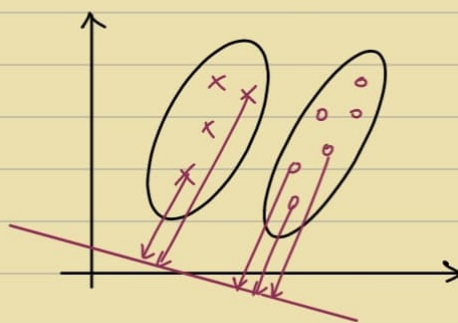
Linear : PCA ; $x' = Wx$

Non-linear : Autoencoder NN ; $x' = f(W, x)$

PCA : Unsupervised



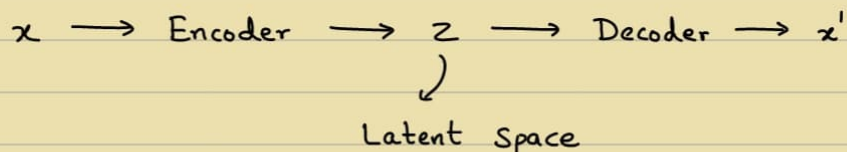
When performing DR



When performing Classification

- DR on Manifolds : LLE , ISOMAP
- Why DR ?
 - (i) Less parameters
 - (ii) Less Compute $\Rightarrow$ Faster
 - (iii) Better Visualization
 - (iv) No Overfitting

• Non-Linear DR : [Autoencoder]



[Visualization : t-SNE]

$\rightarrow$ Low Rank Approximation :

- SVD

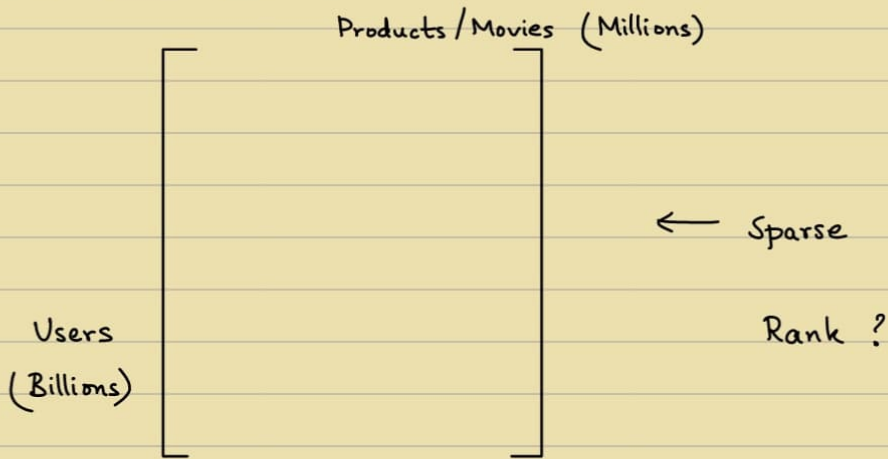
$$A_{m \times n} = \begin{bmatrix} \text{---} \\ \text{---} \\ U_k \\ \text{---} \end{bmatrix} \begin{bmatrix} D_k \\ \text{---} \end{bmatrix} \begin{bmatrix} \text{---} \\ \text{---} \\ V_k^T \\ \text{---} \end{bmatrix}$$

$U_{m \times n}$
 $\downarrow$
 $U_{k \times k}$

$D_{n \times n}$
 $\downarrow$
 $D_{k \times k}$

$V_{n \times n}^T$
 $\downarrow$
 $V_{k \times k}^T$

- Low Rank Matrices:



If all Users identical $\Rightarrow$ Rank = 1

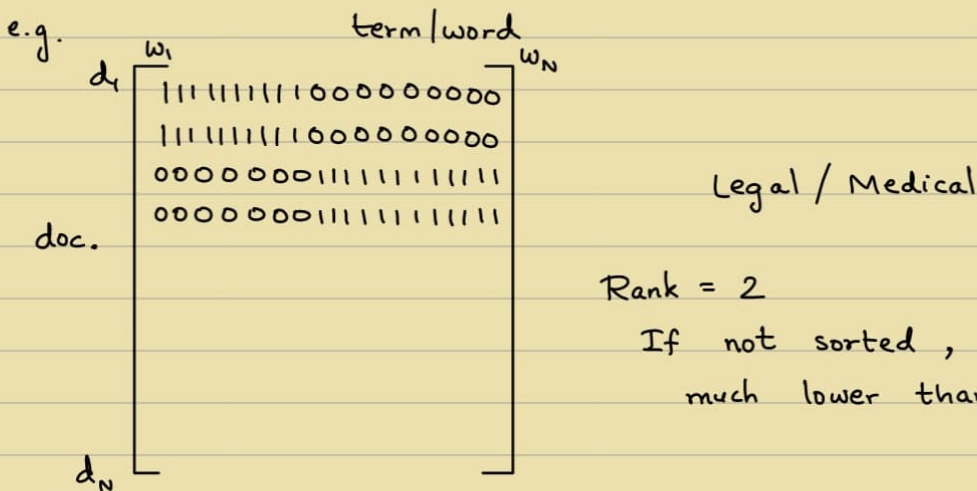
(OR) all Movies identical $\Rightarrow$ Rank = 1

If all Users Random $\Rightarrow$ Rank = $\min(\text{users}, \text{movies})$

(OR) all Movies Random $\Rightarrow$ Rank = $\min(\text{users}, \text{movies})$

↳ Both are impractical, Actual rank - somewhere in b/w

Customer segmentation — Not billions



U_k : Any vector of size billion can be projected onto k dimensions
 $\therefore$ Can find Similar movies
 $\therefore$ Can recommend

Similarly for V^T : Million dimensions (Movie) $\rightarrow$ k dimensions
 $\hookrightarrow$ Very low compared to Million

In the latent space, Problem is well behaved

- Synonymy & Polysemy

One word has diff. meaning

One - Hot Encoding



Query document : $d_q \rightarrow k$ -dimension

Diagonal : Sparse $\Rightarrow$ Rank = 0

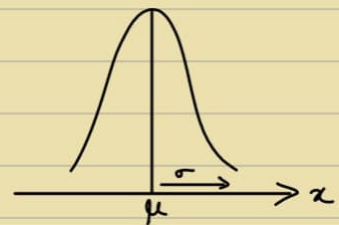
$\therefore L_0$ is best, but dangerous $\because$ We don't know how to use
 $\therefore$ Better option : L_1

5/2

$\rightarrow$ Normal distribution :

- $x \in \mathbb{R}$, $N(\mu, \sigma^2)$

$$P(x|\mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \cdot e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$



- $x \in \mathbb{R}^d$, $N(\mu, \Sigma)$

$$P(x|\mu, \Sigma) = \frac{1}{(2\pi)^{d/2} \cdot |\Sigma|^{1/2}} \cdot e^{-\frac{1}{2}([x-\mu]^T \Sigma^{-1} [x-\mu])}$$

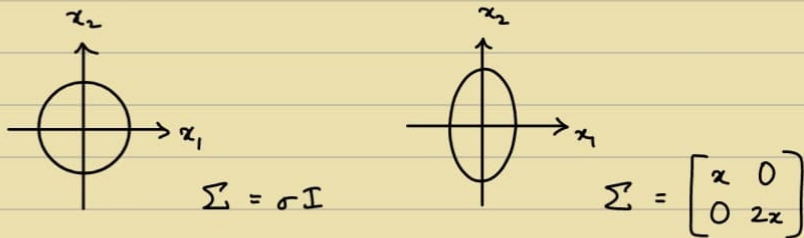
$$\Sigma = \frac{1}{N} \sum_{i=1}^N [x_i - \mu][x_i - \mu]^T \quad \leftarrow \text{Symmetric, PSD}$$
$$z^T \Sigma z \geq 0$$

PSD?

$$z^T \frac{1}{N} \sum_{i=1}^N [x_i - \mu][x_i - \mu]^T z$$

$$= z^T [x_i - \mu][x_i - \mu]^T z \quad \therefore \text{PSD}$$

- Covariance Matrix :



- Bayes Rule :

$$P(x, w_i) = P(x|w_i) \cdot P(w_i)$$

$$= P(w_i|x) \cdot P(x)$$

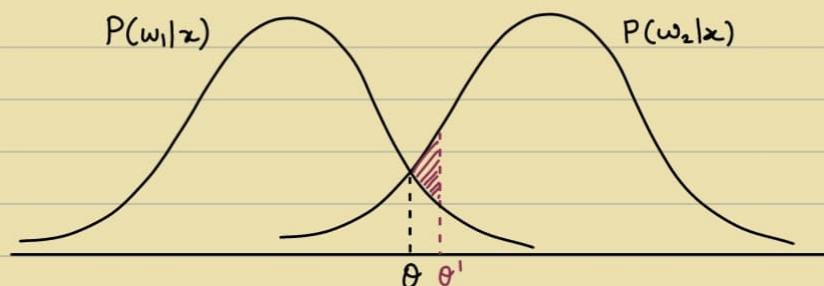
$$P(w_i|x) = \frac{P(x|w_i) \cdot P(w_i)}{P(x)}$$

$$= \frac{P(x|w_i) \cdot P(w_i)}{\sum_{j=1}^c P(x|w_j) \cdot P(w_j)}$$

$$\left\{ \begin{array}{l} P(w_i|x) : \text{Posterior} \\ P(w_i) : \text{Prior} \\ P(x|w_i) : \text{Likelihood} \end{array} \right\} \quad \text{Posterior} \propto \text{Prior} \times \text{Likelihood}$$

$$P(w_1|x) \geq P(w_2|x) \quad \leftarrow \text{Better Classifier compared to } P(w_1) \geq P(w_2)$$

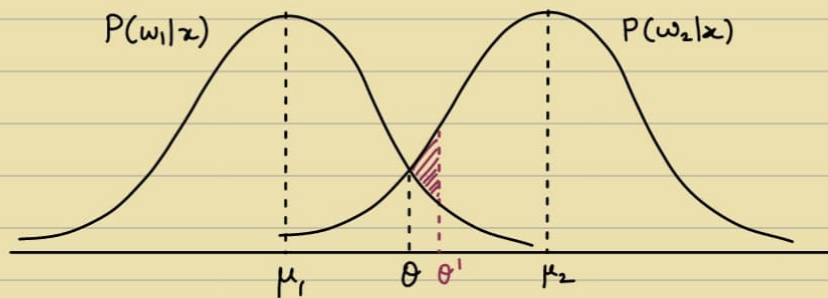
$$\Rightarrow P(x|w_1) \cdot P(w_1) \geq P(x|w_2) \cdot P(w_2)$$



Decide as w_1 if $x \leq \theta$, else w_2

Note : Bayesian Decision - Optimal, Best Classifier (But Given distribution)

But not used everywhere $\because$ Not all data is Gaussian



$$\sigma_1 = \sigma_2$$

$$\theta = \frac{\mu_1 + \mu_2}{2}$$

$$(x - \mu_1)^2 \leq (x - \mu_2)^2$$

Multivariate gives line instead of Point.

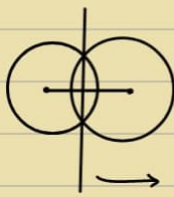
$$e^{\frac{1}{2} [x - \mu_1]^T \Sigma^{-1} [x - \mu_1]} \geq e^{\frac{1}{2} [x - \mu_2]^T \Sigma^{-1} [x - \mu_2]}$$

$$\Rightarrow [x - \mu_1]^T \Sigma^{-1} [x - \mu_1] \geq [x - \mu_2]^T \Sigma^{-1} [x - \mu_2]$$

$$\Rightarrow x^T w_1 + \theta_1 \geq x^T w_2 + \theta_2$$

$$x^T w + \theta \geq 0$$

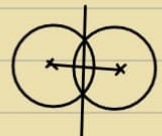
$$\hookrightarrow \text{sign}(w^T x)$$



Perpendicular Bisector



Ellipsoid



$$\cancel{x^T \Sigma^{-1} x} - 2x^T \Sigma^{-1} \mu_1 + \underbrace{\mu_1^T \Sigma^{-1} \mu_1}_{\theta_1} \geq \cancel{x^T \Sigma^{-1} x} - 2x^T \Sigma^{-1} \mu_2 + \underbrace{\mu_2^T \Sigma^{-1} \mu_2}_{\theta_2}$$

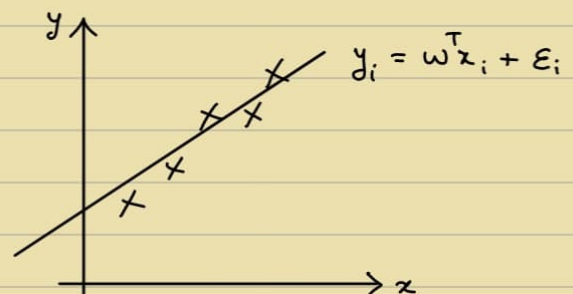
$$\Rightarrow 2x^T \Sigma^{-1} [\mu_1 - \mu_2] + \underbrace{(\theta_1 - \theta_2)}_{\theta} \geq 0$$

• LR :

$$y = w^T x + \varepsilon_i$$

$$L(w) = \prod_{i=1}^N \frac{1}{\sigma \sqrt{2\pi}} \cdot e^{-\frac{(y_i - w^T x_i)^2}{2\sigma^2}}$$

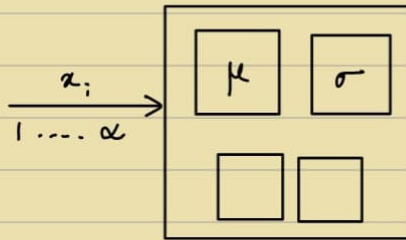
$$\text{Max } L(w) \Rightarrow \text{Max } \log(L(w))$$



$$l(w) = k \cdot \frac{1}{2} \cdot \sum_{i=1}^N (y_i - w^T x)^2 \quad [k : \text{Const.}]$$

$$\min_w J(w) = \min \sum_{i=1}^N (y_i - w^T x)^2 \quad \leftarrow \text{MSE}$$

• Streaming data:



$$\textcircled{1} \mu = x_1$$

$$\textcircled{2} \mu = \frac{\mu + x_2}{2}$$

$$\textcircled{3} \mu = \frac{2\mu + x_3}{3}$$

$$\mu_{N+1} = \frac{(\mu_N \times N) + (1 + x_{N+1})}{N+1}$$

$$\sigma^2 = \frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2$$

$$= \frac{1}{N} \sum_{i=1}^N (x_i^2 - 2\mu x_i + \mu^2)$$

• Feature Selection: [Hard]

$$x' = WX$$

↳ Size: 1000

Want to select 10 best

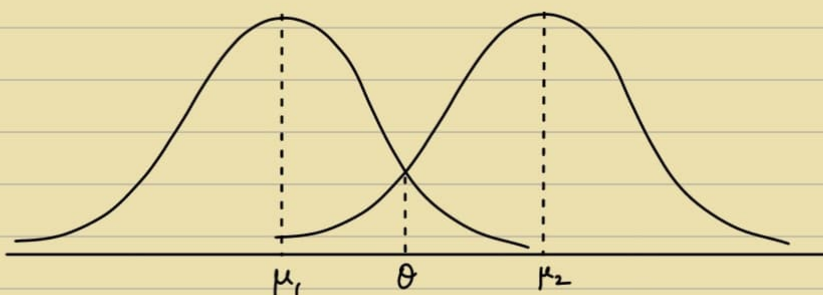
Dimensionality Reduction: Cannot be used $\because$ Changes features
e.g. Lasso Regularization

Generally done: Greedy Algo.

Rank each feature based on $\frac{\mu_A^j - \mu_B^j}{(\sigma_A^j)^2 + (\sigma_B^j)^2}$

[KL Distribution?]

9/2



$$D = \{x_1, x_2, \dots, x_n\}$$

Q) Compute Mean

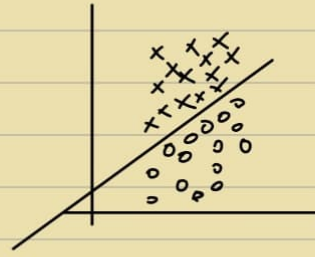
|||

Q) Which distribution

$$N(\mu, \sigma^2) - \mathbb{D}$$

$$N(\mu, \Sigma) - \text{Multivariable}$$

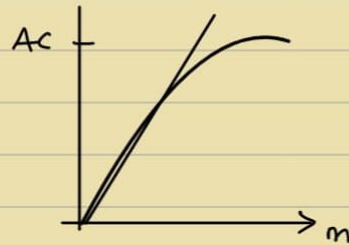
$$x_i \in \mathbb{R}^d$$



Note: PAC

(Probability Approximate Learning)

$$m > f\left(\frac{1}{\epsilon}, \frac{1}{\delta}\right)$$



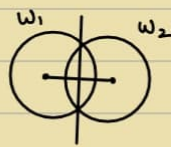
$$\text{Posterior} = \frac{\text{Prior} \times \text{Likelihood}}{\text{Evidence}}$$

const. @ Comparison

$$P(w_1|x) \geq P(w_2|x)$$

$$\Sigma_\ell = \sigma^2 I$$

$$\Sigma_\ell = \Sigma$$

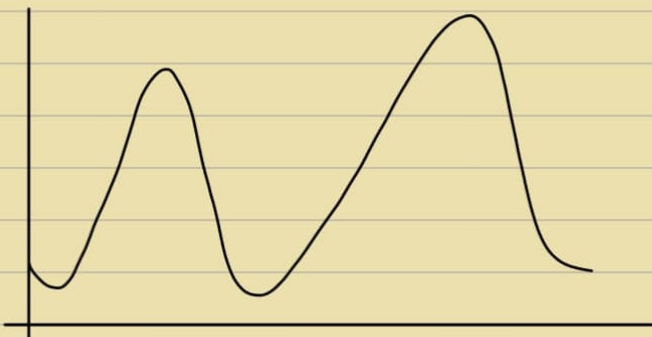


Linear



Quadratic

→ Multimodal Curves:



	P_H	P_T	
C_1	0.5	0.5	- 0.0312
C_2	0.7	0.3	- 0.01323
C_3	0.2	0.8	- 0.02048

Came from $N(\mu_1, \sigma_1^2)$ & $N(\mu_2, \sigma_2^2)$

- MLE - $\arg \max_{\text{model}} P(\text{Data} | \text{Model})$

$$\theta^* = \arg \max_{\theta} P(X | \theta)$$

Q) Given $X = \{X_1, X_2, \dots, X_n\}$

Find μ, σ^2 corresponding to N

Log-likelihood - Easier

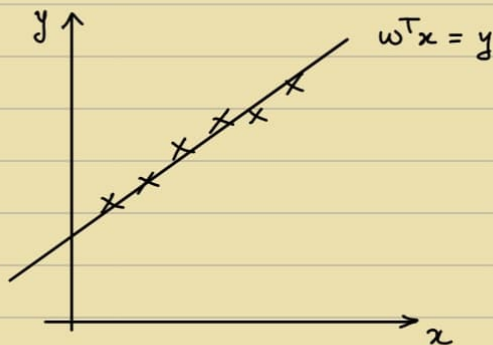
$$\text{e.g. } P(x | \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

Apply MLE,

$$\mu = \frac{1}{N} \sum_{i=1}^N x_i \quad \left| \quad \sigma^2 = \frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2\right.$$

For LR:

MLE $\Rightarrow$ MSE



$$y = w^T x + \epsilon_i$$

$\epsilon_i \sim N(0, \sigma^2)$

GMM - Complex Multi-modal distribution

$$\sum_{k=1}^k \pi_k \cdot N(\mu_k, \sigma_k^2)$$

$\swarrow$ Mixing $\searrow$ θ



If we know mixing, we can find Θ .

If we know Θ , we can find mixing.

$$\Theta = \{\mu, \sigma^2\}$$

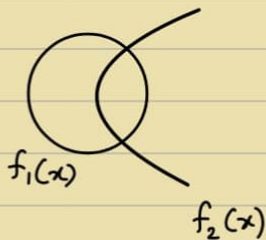
Fixed Point Iteration - Proof of convergence (not in course's scope)

Risk factor: $\lambda_1 P(w_1|x) \geq \lambda_2 P(w_2|x)$ - Decide as w_1 , else otherwise

→ K-Means: Clustering Algo

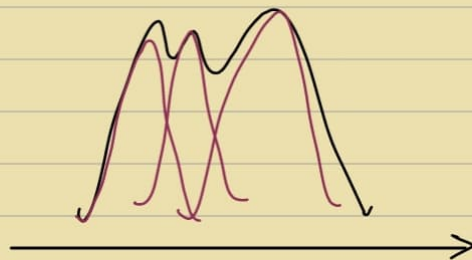
→ K-Means, Naive Bayes and Logistic Regression:

•



$$\max f_i(x)$$

• Component distribution:



• GMM Algo:

$$\gamma_k(x) \in [0, 1]$$

$$\pi_j = \frac{1}{N} \sum_{i=1}^N \gamma_j(x_i)$$

↪ % of Samples of the j^{th} component to the total

Assume C_k a set with $\gamma_k(x_i) = 1 \Rightarrow C_k \subseteq \mathcal{X}$
↪ $\{x_1, x_2, \dots, x_N\}$

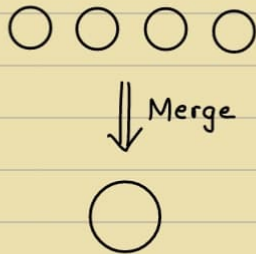
μ_j = mean of C_j^{th} subset

Σ_j = Covariance of C_j^{th} subset

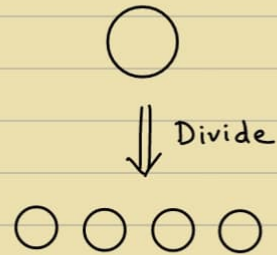
$$X = C_1 \cup C_2 \cup \dots \cup C_k \text{ s.t. } C_i \cap C_j = \emptyset$$

if μ_j & Σ_j are binary $\Rightarrow$ K-Means

(i) Agglomerative :



(ii) Divisive



e.g. Create MST and remove longest edge.



e.g. Hashing - Hash functions

LSH : Locality Sensitive Hashing

ANN : Approximate Nearest Neighbour $\Rightarrow$ Better than KNN

• K-Means :

$$\min_i \sum_i \sum_k \|x_i - \mu_k\|_2^2, \quad x_i \in C_k$$

μ_k : Mean of cluster C_k

Randomly partition of set into K Subsets

Compute means of these subsets as $\mu_1, \mu_2, \dots, \mu_k$

Reassign every sample to the set where mean is closest

Repeat above 2.

$$J(\) = \sum_i \sum_k \|x_i - \mu_k\|^2$$

e.g. $C_1 = \{2, 5, 3, 16, 5\}$

$C_2 = \{1, 4, 18, 22, 20\}$

$\mu_1 = 8.2$

$\mu_2 = 13$

$C_1 = \{1, 2, 3, 4, 5\}$

$C_2 = \{15, 16, 18, 20, 22\}$

e.g. $C_1 = \{A(1,1), B(1.5,2), C(3,4)\} \Rightarrow \mu_1 = (1.83, 2.33)$
 $C_2 = \{D(5,7), E(3.5,5), F(4.5,5)\} \Rightarrow \mu_2 = (4.33, 5.67)$



$C_1 = \{A(1,1), C(3,4), D(5,7)\} \Rightarrow \mu_1 = (3, 4)$
 $C_2 = \{B(1.5,2), E(3.5,5), F(4.5,5)\} \Rightarrow \mu_2 = (3.167, 4)$

• Naive Bayes:

$$P(y|x) = \frac{P(x|y) \cdot P(y)}{P(x)}$$

$$x_1, x_2, \dots, x_d \quad y \in \{0, 1\}$$

$$\hookrightarrow \{0, 1, 2, \dots\}$$

We assume features are conditionally independent given class y

$$P(x_1, x_2, \dots, x_d | y) = \prod_{j=1}^d P(x_j | y)$$

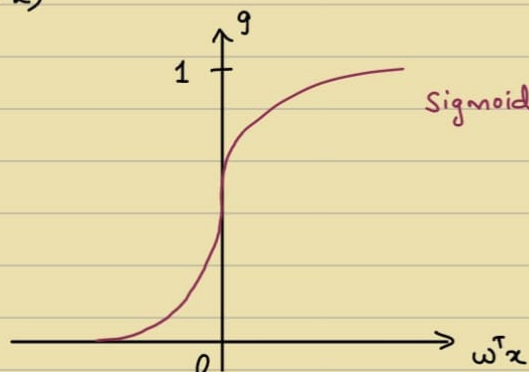
$$\Rightarrow P(x^1 x^2 \dots x^d | y) = \prod_{i=1}^d P(x^i | y)$$

$$P(x_1, x_2, \dots, x_d) \leftarrow \text{Not Practical}$$

$\therefore$ Use conditional independence - Naive Independence

• Logistic Regression:

- Classification Algo
 - Logistic function / Sigmoid
- $\text{sign}(w^T x)$



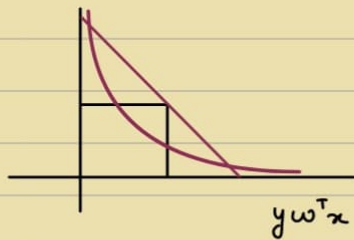
$$g(x) = \frac{1}{1 + e^{-x}}$$

$$1 - \frac{1}{1 + e^{-x}} = \frac{1}{1 + e^x}$$

$$P(y|x) = g(w^T x)^y (1 - g(w^T x))^{1-y}$$

Binary Cross Entropy : $-[y \log p + (1-y) \cdot \log(1-p)] \leftarrow \text{MLE}$

LR with $y \in \{-1, 1\} \rightarrow$ Equation becomes compact

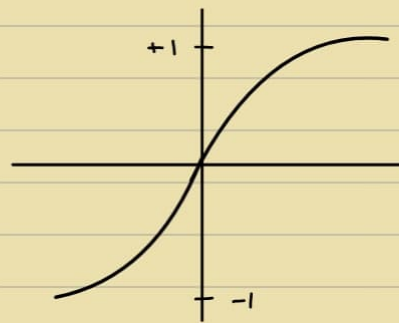


$$\begin{aligned} \omega^T x > 0 & \quad y = +1 \\ \omega^T x \leq 0 & \quad y = -1 \end{aligned}$$

$$y(\omega^T x) \geq 0$$

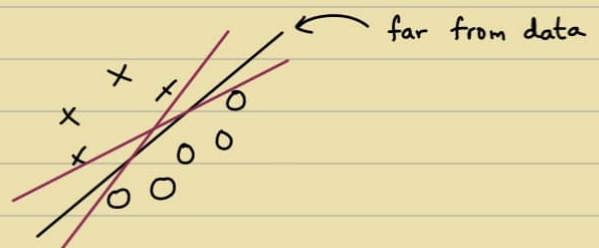
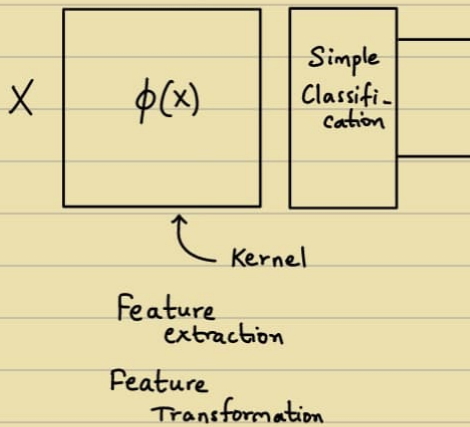
$\arg \max_i p(y=i|x) \leftarrow$ Softmax

$$P(y=c|x; \omega_1, \omega_2, \omega_k) = \frac{e^{\omega_c^T x}}{\sum_{i=1}^k e^{\omega_i^T x}}$$



16/2

$\rightarrow$ SVMs :



- (1) minimise no. of misclassification
- (2) Find a line S.T.

Constraints X is on +ve side
O is on -ve side

(Constr. set)

Hinge loss / SVM loss :

$$L = \max(0, 1 - yf(x))$$

* Maximise Margin b/w 2 lines S.T.

All samples are atleast unit dist.

$$x^T \omega + b = 1$$

$$\text{sign}(\omega^T x + b)$$

$$f(x_i) = x_i \cdot \omega + b$$

$$\therefore x_i \cdot \omega + b \geq +1 \quad \text{for } y_i = +1$$

$$x_i \cdot \omega + b \leq -1 \quad \text{for } y_i = -1$$

$$x_i \cdot \omega \equiv \omega^T x$$

$$\text{Origin to } ax + by + c = 0 : \frac{c}{\sqrt{a^2 + b^2}}$$

$$\text{for } \omega^T x + b = +1 : \frac{1-b}{\|\omega\|}$$

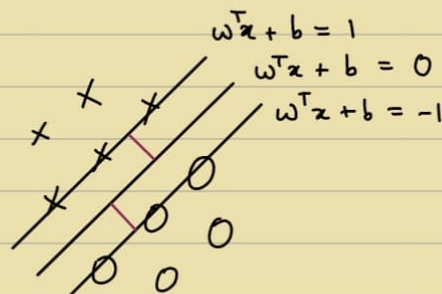
$$\omega^T x + b = -1 : \frac{-1-b}{\|\omega\|}$$

$$\left. \begin{array}{l} \text{for } \omega^T x + b = +1 : \frac{1-b}{\|\omega\|} \\ \omega^T x + b = -1 : \frac{-1-b}{\|\omega\|} \end{array} \right\} \text{Margin} = \frac{2}{\|\omega\|} = \frac{2}{\omega^T \omega}$$

$$y_i(\omega \cdot x_i + b) - 1 \geq 0 \quad \forall i$$

$$\text{Objective : } \min. \frac{1}{2} \omega^T \omega \quad \text{s.t.} \quad y_i(\omega \cdot x_i + b) - 1 \geq 0 \quad \forall i$$

$$y_i \in \{-1, 1\}$$



Support Vectors (α is non-zero)
 α is zero for non-support vectors

$$\text{Dual : } J_d(\alpha) = \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j x_i^T x_j$$

$$\Rightarrow \omega = \sum_{i=1}^N \alpha_i y_i x_i$$

$$b = \frac{1}{|S|} \sum_{i \in S} (y_i - \omega^T x_i)$$

During Testing :

$$\text{sign}\left(\sum_i \alpha_i y_i x_i^T x + b\right)$$

Linear non-separable ?

(XOR Problem ?)

Hard Margin X

Soft Margin with small violation ✓

- Add penalty ξ_i & Add to objective

$$x_i \cdot w + b \geq +1 - \xi_i \quad \text{for } y_i = +1$$

$$x_i \cdot w + b \leq -1 + \xi_i \quad \text{for } y_i = -1$$

$$\xi_i \geq 0 \quad \forall i$$

Objective :

$$\frac{1}{2} w^T w + c \sum_i \xi_i \quad (L_1)$$

L_1 : Sparse

$$\frac{1}{2} w^T w + \frac{c}{2} \sum_i \xi_i^2 \quad (L_2)$$

• Kernels :

Dot - Product is scalar independent of Dimension.

$$p = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

$$p^T q = 4 \iff p_1 q_1 + p_2 q_2$$

$$q = \begin{bmatrix} 2 \\ 2 \end{bmatrix}$$

$$\phi(p)^T \phi(q) = ? \quad \text{Without knowing } \phi$$

$$(p^T q)^2 = p_1^2 q_1^2 + p_2^2 q_2^2 + 2 p_1 q_1 p_2 q_2$$

$$\phi: \mathbb{R}^2 \rightarrow \mathbb{R}^3$$

$$= \begin{bmatrix} p_1^2 \\ p_2^2 \\ \sqrt{2} p_1 p_2 \end{bmatrix}^T \begin{bmatrix} q_1^2 \\ q_2^2 \\ \sqrt{2} q_1 q_2 \end{bmatrix}$$

$\downarrow \quad \quad \downarrow$
 $\phi(p) \quad \quad \phi(q)$

$$\text{Kernel : } K(p, q) = (p^T q)^d$$

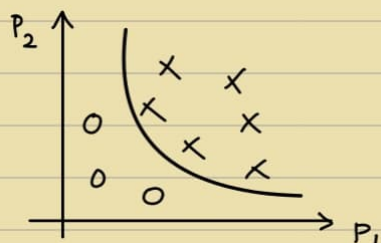
$$K(p, q) = (1 + p^T q)^d$$

$$K(p, q) = \exp(-p^T q)$$

∞ - dimensional space
(Sigmoid, RBF, etc)

Strong feature mapping

Polynomial $\xrightarrow{\text{Kernel}}$ Linear Seperable



$$\begin{bmatrix} p_1^2 \\ p_2^2 \\ \vdots \end{bmatrix}$$

Kernel SVM - Heavy Memory while Testing ($\because$ Carry S.Vectors)
Linear SVM

e.g. $N = 10000$
 $|SV| = 100$

Error estimate = ?

Leave One Out (LOO) $\Rightarrow$ error $\leq \frac{|SV|}{N}$

(Removing non-support vectors does not affect, $\therefore |SV| \leftarrow$ Upper bound)

$K : N \times N \longrightarrow$ Symmetric, Positive Semi-definite
 $K_{ij} = K(x_i, x_j) \rightarrow (x_i^T x_j)^2$

e.g. Kernel PCA

- Hinge Loss curve $\rightarrow$ Convex

Big ϕ & Small classifier vs. Small ϕ and Big Classifier

Hinge Loss - Optimal Solution

Logistic Loss $\Rightarrow$ Hinge Loss

(Primal $\Rightarrow$ Dual?)

VC dimension

- C : Rate of misclassification
 $\hookrightarrow$ Hyperparameters
- SVM - Binary class classification
 $\hookrightarrow$ Long training time

19/2

Q) $\{x_1, x_2, \dots, x_N\}$

$x_i \in \mathbb{R}^d$

Build the best binary classifier

(1) Dimensionality Reduction

Accuracy - better ?

(2) Normalization

$$\|x_i\| = 1$$

$$\mu = 0$$

$$\sigma = 1$$

$\{x\}$ ← Dimension specific

Vector ← usually have subsets

Set

e.g.

$$\begin{bmatrix} \begin{bmatrix} p \end{bmatrix} \\ \begin{bmatrix} q \end{bmatrix} \\ \begin{bmatrix} r \end{bmatrix} \end{bmatrix}$$

Step - I

$$\|p\| = 1$$

$$\|q\| = 1$$

$$\|r\| = 1$$

Step - II

$$\Rightarrow \|x\| = 1$$

Step - III

$$\Rightarrow \{ \text{Set} \}$$

$$\mu = 0$$

- Minimizing only MSE ×

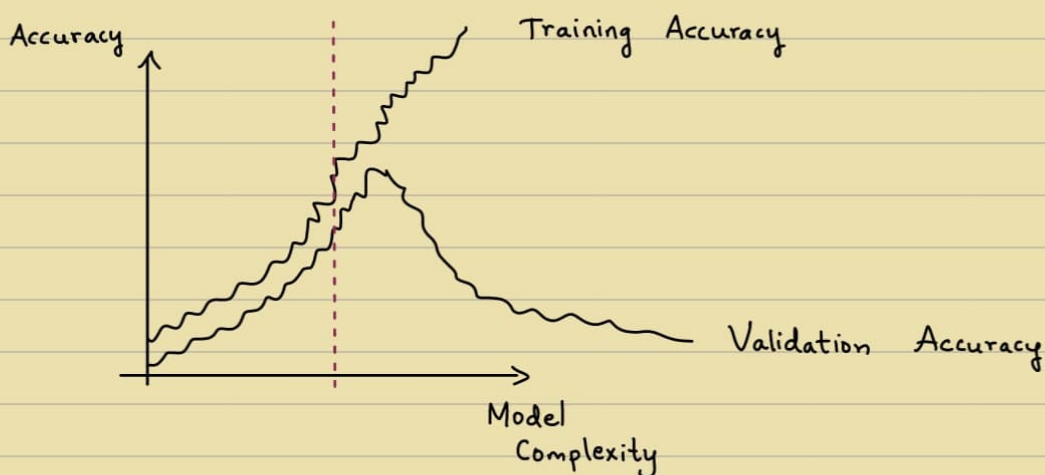
Minimize (MSE + Complexity of sol.)

i.e. Loss + Complexity of solution

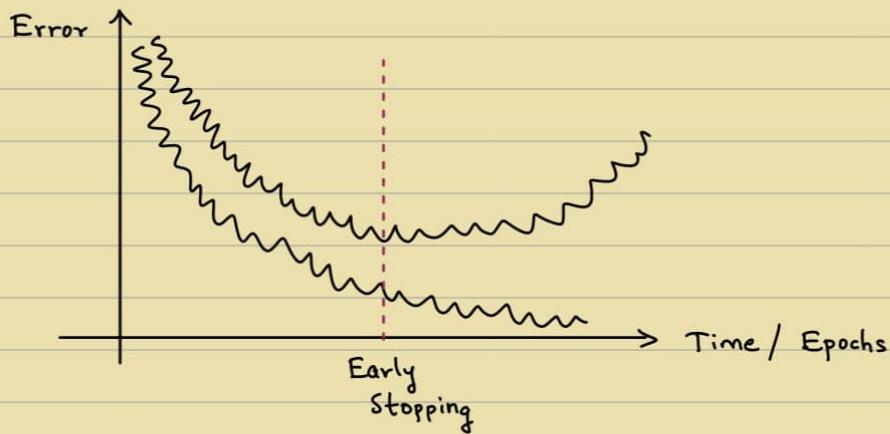
∴ Regularisation is important

Complexity can be degree of polynomial as well.

— Go through all possibilities of model complexities.



$$\omega^{k+1} \leftarrow \omega^k - \eta \cdot \frac{\partial L}{\partial \omega^k}$$



- Overfit : Modeling error
i.e. Train Error $\ll$ Test Error
Train Accuracy $\gg$ Test Error

Avoid Overfitting by

(i) Cross Validation — k subsets : k-1 training & 1 evaluation : k times
↳ Leave One Out (LOO)

- (ii) Train with more data
- (iii) Remove some features
- (iv) Regularisation
- (v) Ensembling
- (vi) Early Stopping

- Train - Train
Val - Internal Test Data
Test - Test

- Bias vs. Variance

↳ Error from sensitivity towards data
↳ Error from wrong assumption

e.g. KNN — $k \uparrow$: Bias $\uparrow$ & Variance $\downarrow$
 $k \downarrow$: Bias $\downarrow$ & Variance $\uparrow$

Decision — Shallow Depth : Bias $\downarrow$
Deep Depth : Bias $\uparrow$

- Parameters vs. Hyperparameters

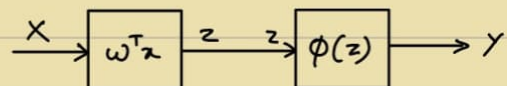
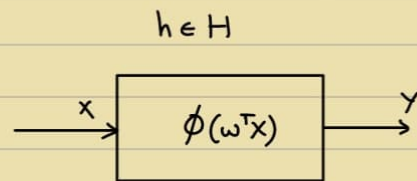
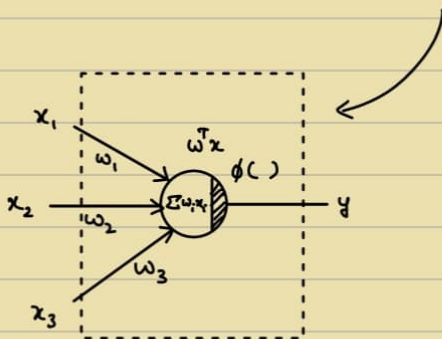
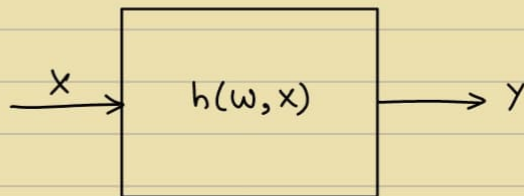
↪ Not changed while training (control how the model learns)
 ↪ Changes while training

9/3

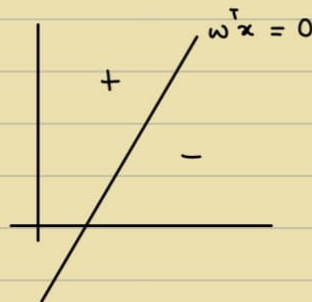
$(x_i, y_i) \leftarrow$ Concept (Data)

$y : f(x) \leftarrow$ Learning

Find the best $h \in H$ that can approximate f on my data D .

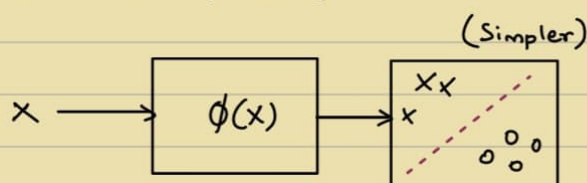


$\phi : \text{sign}(\cdot)$

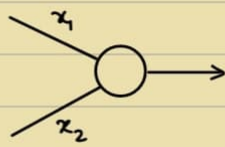


Converges only when data is linearly separable

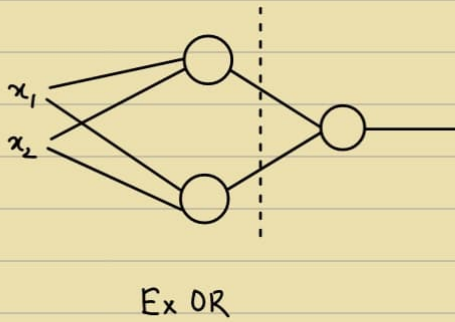
- Ex-OR & Linear Non-separability



$\phi(x) : x \rightarrow x'$
 Kernel ($x = \phi^T \phi$)
 NN



Not enough for XOR
AND/OR ✓



x_1	x_2	A	O	E
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

$$\bar{x}_1 x_2 + \bar{x}_2 x_1$$

Lines $\rightarrow$ Convex Regions
(Overlapping Arbitrary Shapes)

- Multi-Layer Perceptron: (MLP)
(Fully Connected Layers)
- Feed-Forward Neural Network

• Activations:



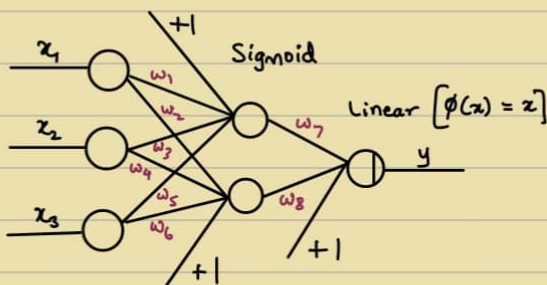
Sign



Sigmoid (0 to 1)
tanh (-1 to 1)

ReLU - Piece-wise Linear
 $\hookrightarrow \max(0, x)$

9)



$$\omega^T z = \begin{bmatrix} \omega_1 \\ \omega_2 \\ \omega_3 \\ \omega_0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ 1 \end{bmatrix}$$

$$\phi(z) = \frac{1}{1 + e^{-z}}$$

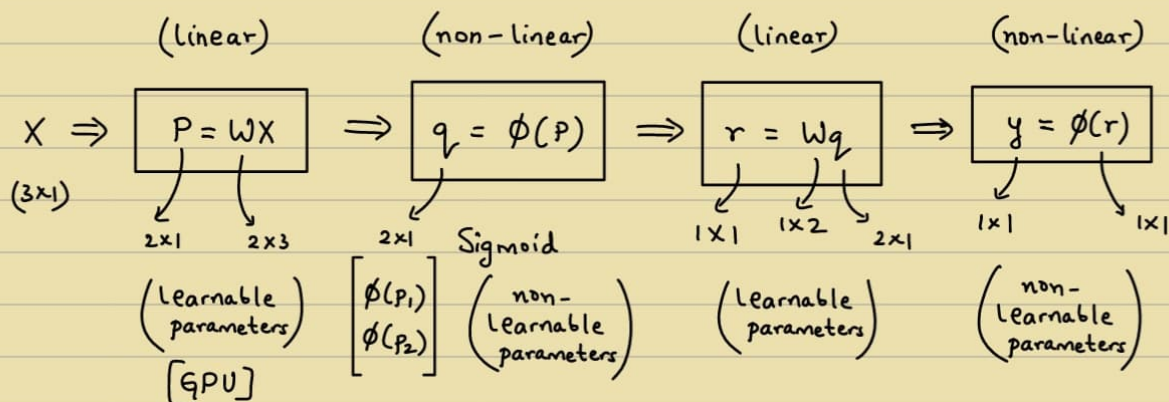
6 + 2 learnable weights without bias
8 + 1 + 1 + 1 learnable weights with bias

$$y = w_7 () + w_8 ()$$

$$= w_7 \left(\frac{1}{1 + e^{-(w_1 x_1 + w_2 x_2 + w_3 x_3)}} \right) + w_8 \left(\frac{1}{1 + e^{-(w_4 x_1 + w_5 x_2 + w_6 x_3)}} \right)$$

$$P = \begin{bmatrix} w_1 & w_2 & w_3 \\ w_4 & w_5 & w_6 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

$$P = WX$$



If :

$$X \Rightarrow P = W_1 X \Rightarrow q = W_2 P \Rightarrow r = W_3 q \Rightarrow r$$

$$Y = W_3 W_2 W_1 X$$

No ReLU $\Rightarrow$ SLP
(if no ϕ)

$\therefore$ Non-Linearity is imp.

$$\text{Sigmoid : } \sigma(z) = \frac{1}{1 + e^{-z}}$$

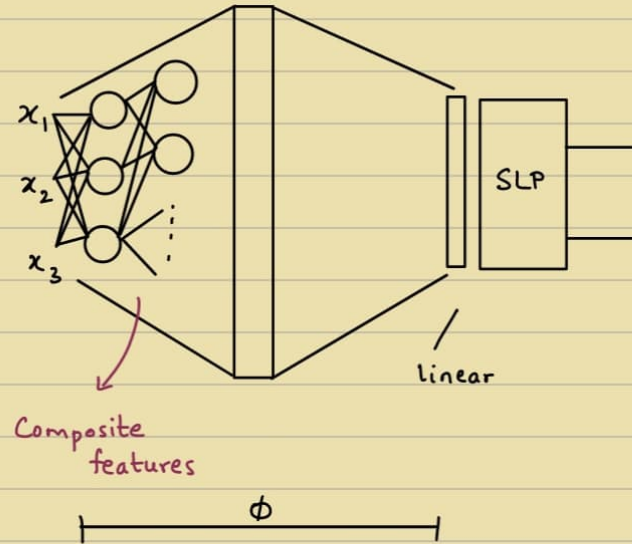
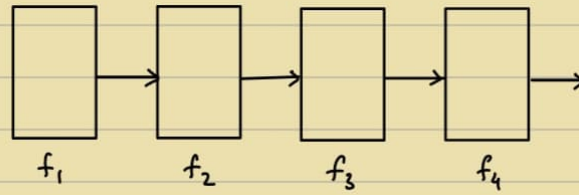
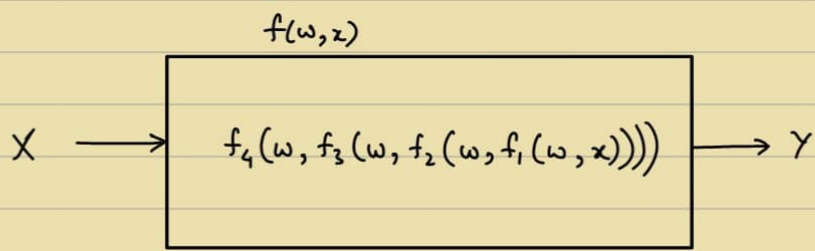
$$\frac{d\sigma(z)}{dz} = \sigma(z)[1 - \sigma(z)]$$

[Similarly, $\tanh(z)$]

Comes handy when backpropagation, Grad. Descent, etc.

Why hidden layers ?

Why activation functions ?



UAT : Any function can be approximated.

- MLP for Regression :

$$y = \phi(x)$$

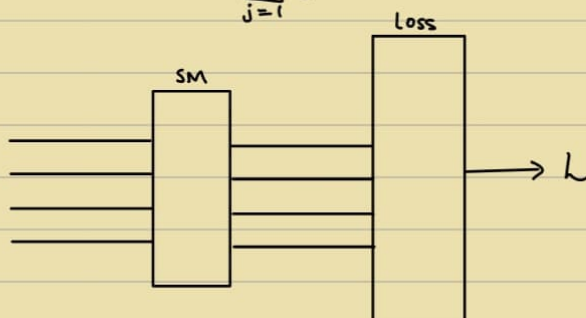
$$y \in \mathbb{R}$$

Linear activations are generally used

Softmax layer :

Convert numbers into probabilities

$$\text{Softmax} : \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}$$



- Backprop :

Initialize weights randomly

Compute x_{p+1}

$$L = \frac{1}{N} \sum_{i=1}^N L(x_{p+1}^i, y_i) \quad \text{full batch}$$

Then, Adjust :

$$\theta^{k+1} \leftarrow \theta^k - \Delta \theta$$

$$\theta^{k+1} \leftarrow \theta^k - \eta \frac{\partial L}{\partial \theta}$$

Loop

$\frac{\partial L}{\partial \theta} \leftarrow$ Expensive

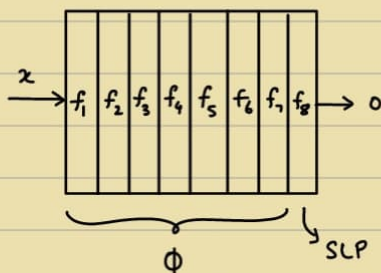
Next class : Backpropagation

$$\frac{\partial L}{\partial w_i} ? \quad \text{for } \phi$$

Loss : compare y_i with $\hat{y}_i$
 $y \sim \hat{y}$

12/3

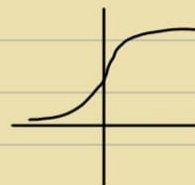
Concatenated blocks



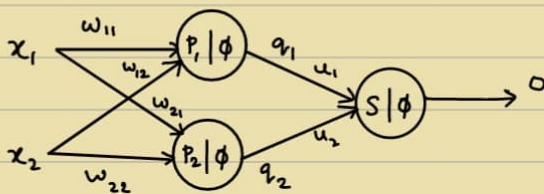
$$o = f_8(f_7(\dots(f_1(x))\dots))$$

Semantic

e.g. $f = \frac{1}{1+e^{-x}}$



e.g.



$$\begin{bmatrix} p_1 \\ p_2 \end{bmatrix} = \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

$$\begin{bmatrix} q_1 \\ q_2 \end{bmatrix} = \begin{bmatrix} \phi(p_1) \\ \phi(p_2) \end{bmatrix}$$

$$s = \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}^T \begin{bmatrix} q_1 \\ q_2 \end{bmatrix}$$

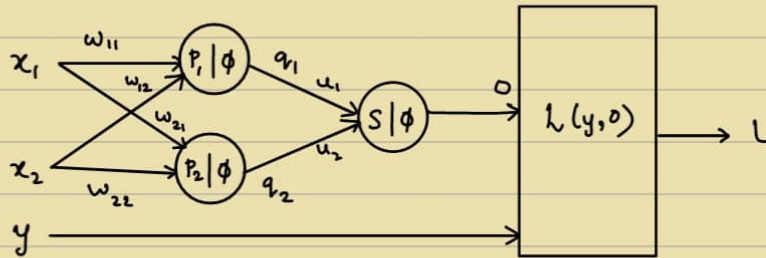
$$p = Wx$$

$$q = \phi(p)$$

$$s = u^T x$$

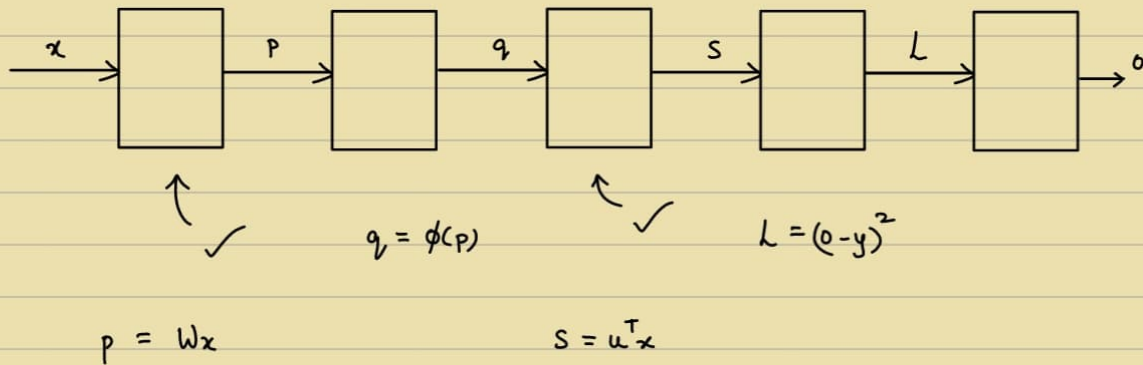
$$o = \phi(s)$$

$$L = L(y, o) = (o - y)^2$$



$$\text{Softmax layer : } \frac{e^{-}}{\sum e^{-}}$$

Learnable parameters : weights



$$\text{As } \omega^{k+1} \leftarrow \omega^k - \eta \frac{\partial L}{\partial \omega^k}$$

$$\frac{\partial L}{\partial u_1} = \frac{\partial L}{\partial o} \cdot \frac{\partial o}{\partial s} \cdot \frac{\partial s}{\partial u_1}$$

$$= 2(o - y) \cdot \phi'(s) \cdot q_1$$

Reusable when $\frac{\partial L}{\partial u_2}$

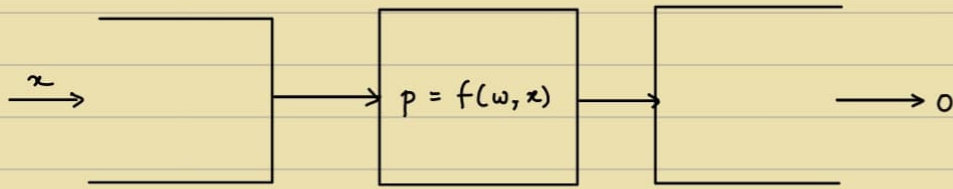
$$\frac{\partial L}{\partial \omega_{11}} = \frac{\partial L}{\partial o} \cdot \frac{\partial o}{\partial s} \cdot \frac{\partial s}{\partial q_1} \cdot \frac{\partial q_1}{\partial p_1} \cdot \frac{\partial p_1}{\partial \omega_{11}}$$

$$= 2(o - y) \cdot \phi'(s) \cdot u_1 \cdot \phi'(p_1) \cdot x_1$$

Reusable when $\frac{\partial L}{\partial \omega_{12}}$

Updating weights - Learning

Long chain of multiplication $\leftarrow$ Vanishing Gradients / Exploding Gradients



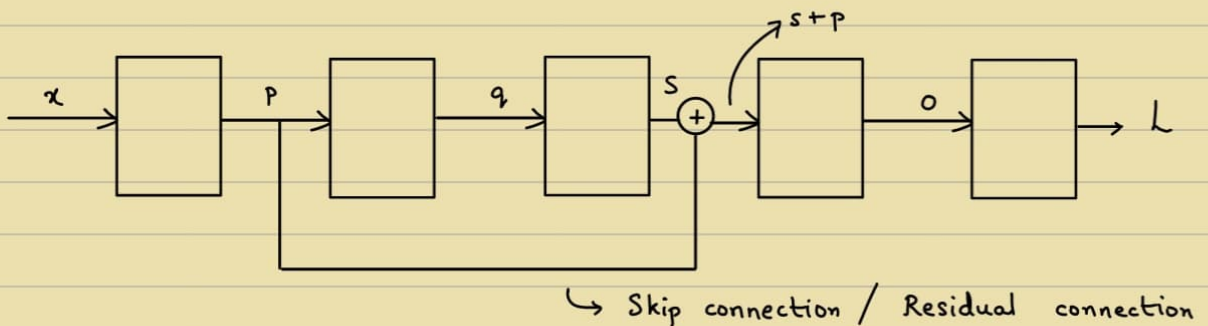
$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial p} \cdot \frac{\partial p}{\partial x} \quad (\text{Layer})$$

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial p} \cdot \frac{\partial p}{\partial w}$$

H.W

Define custom learnable sigmoid layer in PyTorch and train an MLP
 $w \uparrow \Rightarrow$ Steeper

Q)



Find $\frac{\partial L}{\partial w}$

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial o} \cdot \frac{\partial o}{\partial s'} \cdot \frac{\partial s'}{\partial q} \cdot \frac{\partial q}{\partial p} \cdot \frac{\partial p}{\partial w} + \frac{\partial L}{\partial o} \cdot \frac{\partial o}{\partial r} \cdot \frac{\partial r}{\partial p} \cdot \frac{\partial p}{\partial w}$$

$$\left\{ \begin{array}{l} p = wx \\ q = \phi(p) \\ s = u^T x \end{array} \right. \quad \left\{ \begin{array}{l} o = \phi(s+p) \\ L = L(y, o) = (o-y)^2 \end{array} \right.$$

$$\begin{aligned} s+p &= u^T x + wx \\ &= (u^T + w) x \end{aligned}$$

$$\frac{\partial s'}{\partial q} = \frac{\partial s'}{\partial s} \cdot \frac{\partial s}{\partial q}$$

(Solves Vanishing gradient problem)

$$\text{Then, } w^{n+1} \leftarrow w^n - \eta \frac{\partial L}{\partial w}$$

Next class : Build on Backprop. (Grad. Descent)

Neural Network : Approximation of f

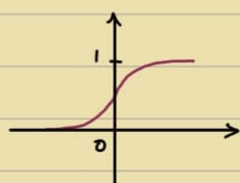
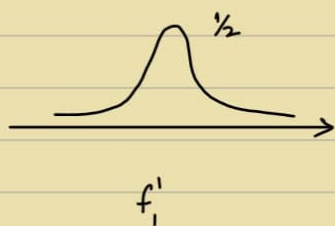
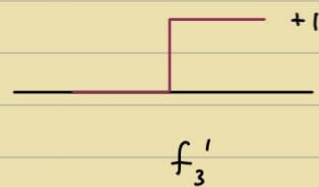
e.g. $y = 4x^2 + 2x$
 $\Rightarrow y = f(x)$

Problems :

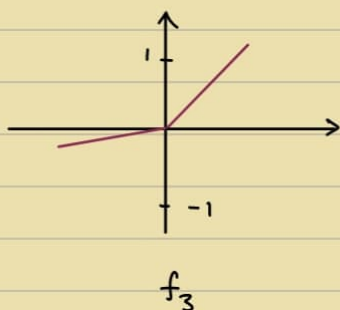
- ① Problem Abstraction
- ② Choice of Architecture
- ③ Training $\leftarrow$ Backpropagation

forward pass : Compute loss

Backward pass : Update Weights

 $\sim 85\%$ - BaselineSigmoid (f_1)tanh (f_2)ReLU (f_3) f_1'  f_3'

Leaky ReLU :

 f_3

It's not about implementing BP
 Can you use BP and train a
 NN ?

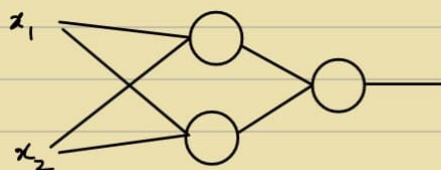


Subgradient ?

- Upgrading BP:

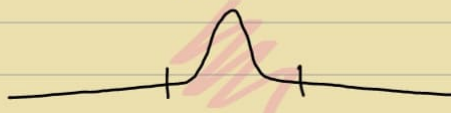
- (1) Better Initialisation

if all weights are equal or zero



If both weights are equal, then outputs will be same. We want them to complement each other.

Random, but weighted sum in this region.



Xavier initialisation

Cunningham initialisation

- (2) Gradient Estimates

We don't use loss, we use $\frac{\partial L}{\partial \omega}$ (Gradient of loss)

Derivative of Loss - Exact or Approximate?

We're looking for the direction of the gradient and not the magnitude - Estimate is fine.

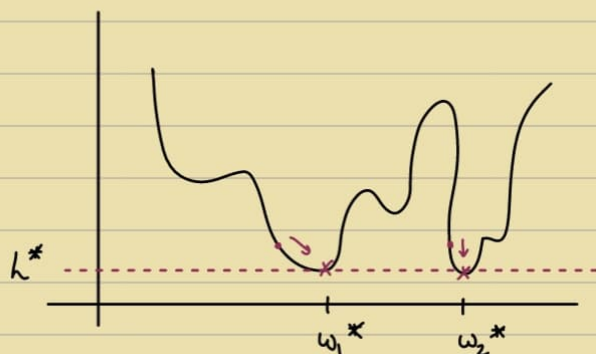
Reasonable estimate in $1/1000^{\text{th}}$ time ✓ $\omega^{(k+1)} \leftarrow \omega^{(k)} - \eta \cdot \frac{\partial L}{\partial \omega}$

Whole batch - Very Expensive

Single Sample - Very efficient but noisy

Stochastic Mini-batch GD - Balance b/w both

e.g. Loss function



Multiple Local Minima

↳ Not a problem

Same performance (approx)

Local Minima - Derivatives of all weights / params. = 0 @ point
↳ Probability is low

Epoch vs. Iteration

Learning Rate - Schedulers

Momentum : $\theta^{k+1} \leftarrow \theta^k - \Delta\theta$

$\Delta\theta = \eta \frac{\partial L}{\partial \theta} + \underbrace{\gamma \Delta\theta^{t-1}}_{\text{Additional term}}$

(3) Regularization :

Data Augmentation - Params to Data ratio $\uparrow$

Add noise - Inputs, Outputs, weights

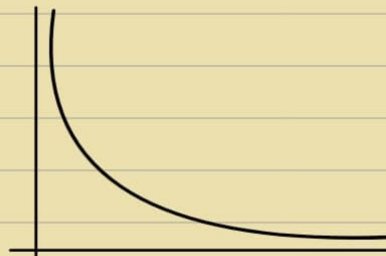
Loss function - L_1, L_2 regularization

- Early stopping - stop when val. acc. $\uparrow$

Will loss = 0 after long time training ? No

Model Complexity

Data Noise



Won't increase and non-zero
Always decreasing.

26/3

- Choose good initialisation
i.e. Take Pre-trained network

Fine-tune : domain exclusive training

- Normalisation - Batch Norm
↳ Layer $\rightarrow \hat{x} = f(w, x)$
- Dropout

- For quadratic loss function, GD Convergence in 1 Step is possible
But, we are truncating from Taylor series

When $f(x) \rightarrow \text{linear} \Rightarrow$ Only Grad. is enough

When $f(x) \rightarrow \text{Quad} \Rightarrow$ Grad. & Hessian is req.

But we are still approximating, $\therefore$ Not exact

$$\text{Optimal } \eta = \frac{\|\nabla J\|^2}{\nabla J^T H \nabla J} \rightarrow \text{Calc. is memory expensive.} \\ [H : d \times d]$$

Memory vs. Time Trade off

$$J(w^{k+1}) = J(w^k) + s^T \nabla J + \frac{1}{2} s^T H s$$

$$\frac{d}{ds} \stackrel{\text{set}}{=} 0 \Rightarrow \text{Newton Method}$$

$$w^{k+1} = w^k - \underbrace{(H + \alpha I)^{-1}}_{\text{Always Invertible}} \nabla J$$

But Inversion is costly $O(d^3)$

$$\alpha \uparrow \Rightarrow \text{G.D.}$$

Steepest GD : Find steep Grad. direction

- Momentum :

η gives diff. η 's for diff. values

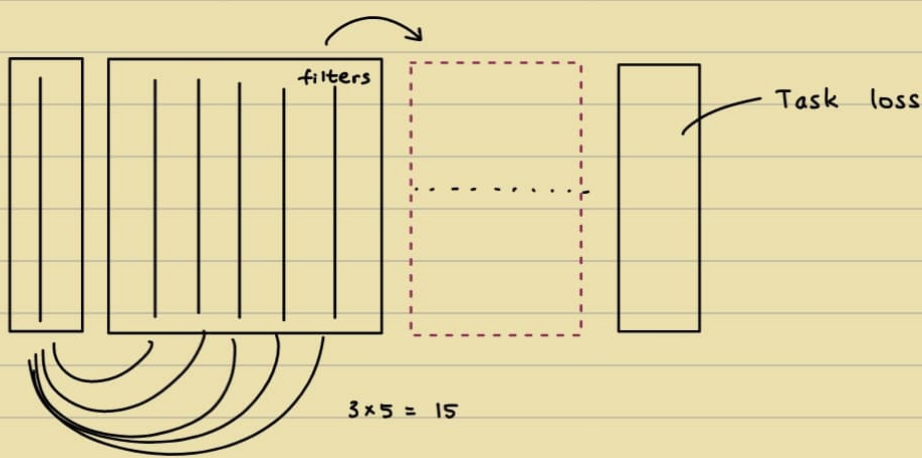
Adam : Momentum + Adaptive Sealing
(+ Bias correction)

2/4

$\rightarrow$ CNN :

$$\text{Convolution : } f * g = \int_{-\infty}^{\infty} f(x) \cdot g(t-x) dx$$

In CNN $\Rightarrow$ Correlation, i.e. Dot Product with Padding



No. of weights in independent of size

Whereas in MLP :

$n \times n \rightarrow n^2$ weights to learn

But in CNN,

Independent of n



No Padding $\Rightarrow$ Output size decreases

- Window size

Pooling

Stride

Pad

Flattening

AlexNet - Oversaturated features

Then, Deeper & Lighter

2 3×3 filters better than 1 5×5 filters

1×1 Kernel Size is used for dimensionality Reduction.

6/4

$\rightarrow$ More on CNNs :

1D Conv. Kernels / Filters

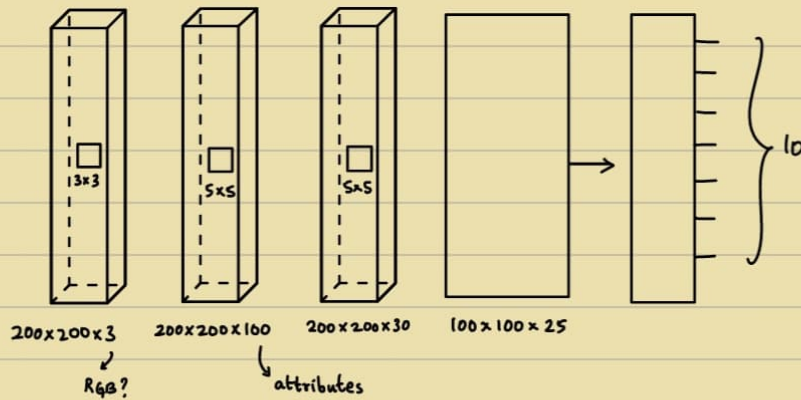
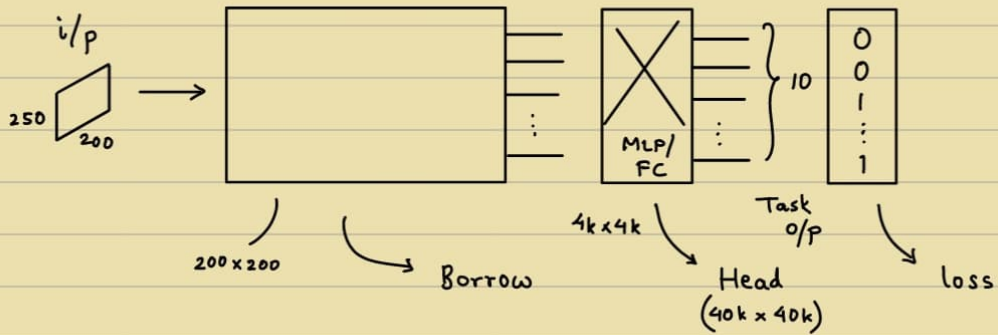
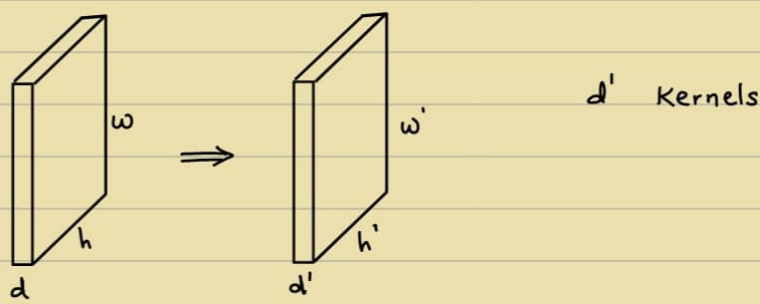
2D Conv. Feature Maps

3D Conv.

Pad

Stride

Pool



Kernel size change, $3 \times 3 \rightarrow 5 \times 5$, Output affected?
 Stride change, $1 \rightarrow 2 \Rightarrow$ Output affected

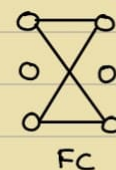
• How many params, filters, etc?

No. of learnable params : $H_k \times W_k \times C_{in} \times C_{out}$

$H \times W \times C_{in} \xrightarrow{\text{(Kernel)}} H_k \times W_k \times C_{out} \rightarrow H' \times W' \times C_{in}$

Image Taken in camera $\rightarrow$ reduced to $200 \times 200 \times 3 = 120K$

Locally connected layer
is better



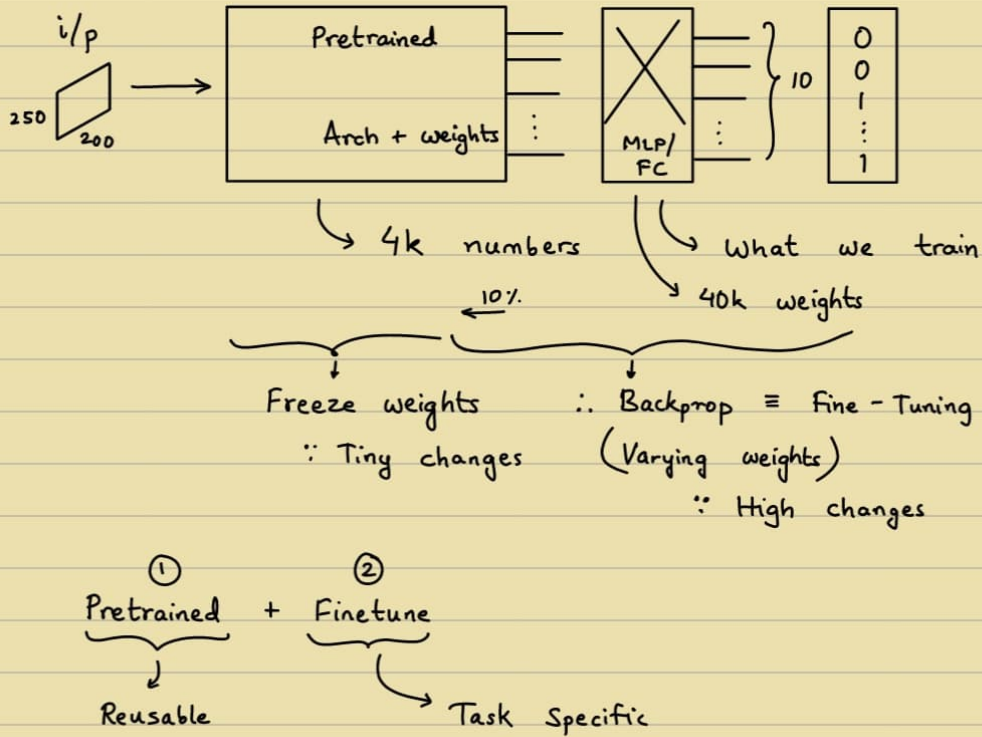
$120K \times 120K = 144B$

Weight Sharing $\leftarrow$ Reusable weights

$$100 \times 3 \times 3 \times 3 = 2700 \text{ learnable weights}$$

$$\therefore 14.4 \text{ M} \rightarrow 2700 \text{ weights}$$

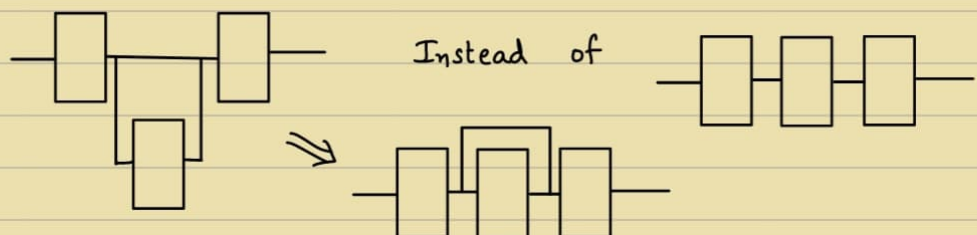
Borrow Arch. and weights (trained)



Task domain vs. Pretrained domain

Early / Lowest layers learn highly common patterns
& Also converge quicker
e.g. Shapes, Colours

Training data, Deep network performs bad
Vanishing Gradient Problem
Sol.: Residual / Skip connections



Extra layers $\equiv$ Identity layers



$$\begin{aligned}\frac{\partial h}{\partial x} &= \frac{\partial h}{\partial y} \cdot \frac{\partial y}{\partial x} \\ &= \frac{\partial h}{\partial y} \cdot (1 + F'(x))\end{aligned}$$

RESNET

DENSENET